KPR Institute of Engineering and Technology

Department of Artificial Intelligence and Data Science

U21ADP05 - Exploratory Data Analysis and Visualization

Assignment II: Case Study

Exploratory Data Analysis and Visualization of Bank Telemarketing Campaigns: Statistical Insights and a Deep Learning Model for Term-Deposit Subscription

Item	Detail
Student name	Dharshan
Roll number	23AM019
Department	Department of Artificial Intelligence and Machine Learning
Batch / Year-Semester	2023-2027 / V Semester (Academic year 2026-2027, odd semester)
Course coordinator	Mr. Rushikesh Kadam
Date of submission	15 October 2026
Dataset	UCI Bank Marketing (Moro, Cortez & Rita, 2014)
Code repository	github.com/<your-github-id>/bank-marketing-eda-dl (replace with your public link)

Abstract

Telemarketing is costly, so banks need to know which clients are worth calling. This study explores the UCI Bank Marketing dataset (45,211 calls, 16 client and campaign features), where 11.7% of calls end in a term-deposit subscription. Following the seven stages of visualization, the data were cleaned, and distributions, associations and 22 figures were analysed. Contact counts were over-dispersed (Negative Binomial beat Poisson) and call duration was closer to lognormal than exponential. A previously successful campaign (64.7% conversion) and the calling month were the strongest signals, and age showed a U-shape. A leakage-free multilayer perceptron using pre-call information reached a test ROC-AUC of 0.784 (PR-AUC 0.415), above logistic regression (0.759) but below gradient boosting (0.801).

Keywords: exploratory data analysis, data visualization, probability distributions, Negative Binomial, bank telemarketing, multilayer perceptron, data leakage.

Contents

1. Introduction and Objectives
2. Dataset Description
3. EDA and Preprocessing (including the seven visualization stages)
4. Data Visualization (Figures 1-14, statistical analysis, key findings)
5. Deep Learning Model
6. Results Visualization and Interpretation (Figures 15-22)
7. Conclusion and Future Scope
8. References
9. Appendix (rubric matrix, reproducibility, source code)

1. Introduction and Objectives

Direct marketing by telephone is one of the most common ways for banks to sell long-term deposit products, but it is expensive: most calls fail, and repeated calls irritate clients. Understanding which clients, timings and campaign conditions lead to a subscription lets a bank focus scarce agent time on promising prospects. This case study applies exploratory data analysis (EDA), statistical modelling of distributions, and visualization to a real campaign dataset, and then builds a deep learning model that predicts whether a client will subscribe to a term deposit.

1.1 Problem statement

Given information about a client and the campaign that is available before the call, which factors are associated with a successful subscription, and how well can a neural network rank clients by their probability of subscribing? The target variable y is binary and only 11.7% of calls are successful, so the problem is imbalanced and accuracy alone is not an adequate performance measure.

1.2 Objectives

- Acquire a real-world public dataset with at least 15 features and document its provenance, structure and limitations (Acquire, Parse).
- Detect and treat hidden missing values, a sentinel code, duplicates, outliers and skewed variables with justified, leakage-free preprocessing (Filter).
- Quantify univariate, probability-distribution, bivariate and multivariate behaviour with appropriate statistics and tests (Mine).
- Communicate the findings with at least five well-designed, interpreted visualizations covering all seven stages, including an interactive explorer (Represent, Refine, Interact).
- Build, tune and honestly evaluate at least one deep learning model, compare it with baselines, and examine leakage and temporal robustness.
- Draw practical conclusions and state the limitations and future scope, with a reproducible GitHub repository.

1.3 Literature context

The dataset was introduced by Moro, Cortez and Rita (2014), who modelled bank telemarketing success with logistic regression, decision trees, neural networks and support vector machines, and used sensitivity analysis of the neural network to extract business knowledge. The seven-stage framework (acquire, parse, filter, mine, represent, refine, interact) is from Fry (2007), and the emphasis on looking at data before modelling follows Tukey (1977). For imbalanced outcomes, precision-recall analysis is more informative than ROC alone (Saito & Rehmsmeier, 2015), and benchmark evidence suggests tree ensembles often remain competitive with deep networks on tabular data (Grinsztajn et al., 2022), which is why gradient boosting is included here as a reference.

2. Dataset Description

2.1 Candidate comparison and selection

Table 1. Dataset candidates (figures taken from UCI documentation; only the selected dataset was downloaded and analysed)

Candidate	Rows	Inputs	Types	Target	Opportunities for CO3-CO5	Decision
Bank Marketing (UCI #222)	45,211	16 + target	7 numeric, 9 categorical	Binary: term-deposit subscription	Counts (campaign, previous), duration (waiting-time-like), sentinel and 'unknown' values, documented leakage variable, time order	SELECTED
Default of Credit Card Clients (UCI #350)	30,000	23 + target	numeric / ordinal	Binary: payment default	Rich numeric history but few natural discrete-count or waiting-time variables	Runner-up
Student Performance (UCI #320)	649 (Portuguese)	30 + grades	mixed	Grade regression / pass-fail	Small sample limits deep learning and distribution fitting	Rejected

Bank Marketing was chosen because it has a clear business application, 16 genuine input fields, enough observations (45,211) for both reliable statistics and a neural network, a mix of data types, and several features that make the probability-distribution analysis meaningful rather than forced: event-like counts (campaign, previous), a waiting-time-like variable (duration) and a binary outcome.

2.2 Source, acquisition and verification

Table 2. Dataset source and verification

Item	Detail
Original source	UCI Machine Learning Repository, dataset 222 'Bank Marketing' - https://archive.ics.uci.edu/dataset/222/bank+marketing
Citation	Moro, S., Cortez, P., & Rita, P. (2014). Decision Support Systems, 62, 22-31. doi:10.1016/j.dss.2014.03.001
File used	bank-full.csv (semicolon separated; ordered by date, May 2008 to November 2010 per UCI documentation)
Download method	src/data_loader.py tries the official UCI archive first, then a GitHub mirror of the same file; the file is validated (shape 45,211 x 17, expected columns, y in {yes, no}).
This submission	The sandbox used to execute the project could not reach UCI, so the verified mirror was used; the file passes all validation checks and matches the UCI description (45,211 instances, 17 fields).
SHA-256 of the analysed file	d1513ec63b385506f7cfce9f2c5caa9fe99e7ba4e8c3fa264b3aaf0f849ed32d
Licence / terms	See the licence statement on the UCI dataset page; cite Moro et al. (2014) when reusing.
Dimensions	45,211 rows x 17 columns = 16 input features + 1 target; no NaN cells; no duplicate rows
Target variable	y: client subscribed to a term deposit (yes/no)

2.3 Feature descriptions and basic statistics

Table 3. Data dictionary (descriptions follow the UCI documentation)

field	type	description	values / range
age	numeric (int)	Age of the client in years	18-95
job	categorical	Type of job (12 levels incl. 'unknown')	admin., blue-collar, management, technician, ...
marital	categorical	Marital status ('divorced' includes widowed)	married, single, divorced
education	categorical	Education level (incl. 'unknown')	primary, secondary, tertiary, unknown

field	type	description	values / range
default	binary	Has credit in default?	yes / no
balance	numeric (int)	Average yearly balance in euros (negative = overdrawn)	-8,019 to 102,127
housing	binary	Has a housing loan?	yes / no
loan	binary	Has a personal loan?	yes / no
contact	categorical	Contact communication type (incl. 'unknown')	cellular, telephone, unknown
day	numeric (int)	Day of the month of the last contact	1-31
month	categorical	Month of the last contact	jan ... dec
duration	numeric (int)	Duration of the last contact in seconds (known only AFTER the call -> excluded from the primary model)	0-4,918
campaign	numeric (int)	Number of contacts performed during this campaign for this client (includes the last contact)	1-63
pdays	numeric (int)	Days since the client was last contacted in a previous campaign (-1 = never contacted)	-1, 1-871
previous	numeric (int)	Number of contacts performed before this campaign for this client	0-275
poutcome	categorical	Outcome of the previous marketing campaign (incl. 'unknown')	success, failure, other, unknown
y	binary (TARGET)	Did the client subscribe to a term deposit?	yes / no

Table 4. Descriptive statistics of the numeric variables (all 45,211 records)

variable	count	mean	std	min	Q1	median	Q3	max	skewness	excess_kurtosis
age	45,211	40.94	10.62	18.00	33.00	39.00	48.00	95.00	0.68	0.32
balance	45,211	1,362.27	3,044.77	-8,019.00	72.00	448.00	1,428.00	102,127	8.36	140.74
day	45,211	15.81	8.32	1.00	8.00	16.00	21.00	31.00	0.09	-1.06
duration	45,211	258.16	257.53	0.00	103.00	180.00	319.00	4,918.00	3.14	18.15
campaign	45,211	2.76	3.10	1.00	1.00	2.00	3.00	63.00	4.90	39.25
pdays	45,211	40.20	100.13	-1.00	-1.00	-1.00	-1.00	871.00	2.62	6.93
previous	45,211	0.58	2.30	0.00	0.00	0.00	0.00	275.00	41.85	4,506.36

Table 5. Categorical variables: number of levels, most frequent level, hidden-missing share

variable	levels	most frequent level	'unknown' share
job	12	blue-collar (21.5%)	0.6%
marital	3	married (60.2%)	-
education	4	secondary (51.3%)	4.1%
default	2	no (98.2%)	-
housing	2	yes (55.6%)	-
loan	2	no (84.0%)	-
contact	3	cellular (64.8%)	28.8%
month	12	may (30.4%)	-
poutcome	4	unknown (81.8%)	81.7%
y	2	no (88.3%)	-

2.4 Limitations of the dataset

- No client identifier or year: repeated clients and campaign phase cannot be separated; month mixes 2008-2010 conditions.
- The file is the older 17-field version of the data; the richer 'bank-additional' version (macro-economic indicators) was not used.
- duration is only known after a call ends, so it is excluded from the primary predictive model (documented by the data owners).

- Observational data from one Portuguese bank: results show association, not causation, and may not transfer to other banks or periods.

3. EDA and Preprocessing

3.1 Data-quality findings (Parse)

Table 6. Data-quality checks on the raw file

column	issue	count	percent
job	'unknown' label	288	0.64
marital	'unknown' label	0	0.00
education	'unknown' label	1,857	4.11
default	'unknown' label	0	0.00
housing	'unknown' label	0	0.00
loan	'unknown' label	0	0.00
contact	'unknown' label	13,020	28.80
month	'unknown' label	0	0.00
poutcome	'unknown' label	36,959	81.75
pdays	sentinel -1 (never contacted)	36,954	81.74
balance	negative balance (valid overdraft)	3,766	8.33
duration	zero-second calls	3	0.0070
(all)	exact duplicate rows	0	0.00
(all)	NaN cells	0	0.00

The raw file contains no NaN cells and no duplicate rows. Missing information is instead encoded as the text label 'unknown' (poutcome, contact, education, job) and as the sentinel pdays = -1. poutcome = 'unknown' (36,959 clients) and pdays = -1 (36,954 clients) overlap in all but 5 records, so they both mean 'not contacted in an earlier campaign'. Negative balances (3,766 clients) are valid overdrafts, and 3 calls lasted 0 s.

3.2 Missing values, duplicates and cleaning decisions (Filter)

Table 7. Cleaning decisions

Issue	Treatment	Justification
NaN values	none present	Nothing to impute.
'unknown' categories	kept as an explicit level	Not random missingness (e.g. contact = unknown converts at 4.1% vs 14.9%); mode-imputation would destroy signal.
pdays = -1	-> flag previously_contacted (0/1) + pdays_clean (0 if never contacted)	Prevents the sentinel being treated as a real 'one day ago' value; deterministic, so no leakage.
Duplicates	none found; no rows removed	Checked with DataFrame.duplicated() (0 rows).
Text standardisation	strip + lower-case categoricals	Guards against inconsistent labels (none were found).
Target	y -> y_bin (1 = yes)	Used for modelling; the original column is kept for tables and plots.

3.3 Outliers (Mine / Filter)

Table 8. IQR outlier screening (1.5 x IQR fences)

variable	lower fence	upper fence	n below	n above	% flagged	n > 3 IQR	max	p99
age	10.5	70.5	0	487	1.1	3	95	71.0
balance	-1,962.0	3,462.0	17	4,712	10.5	2,443	102,127	13,164.9
duration	-221.0	643.0	0	3,235	7.2	1,155	4,918	1,269.0

variable	lower fence	upper fence	n below	n above	% flagged	n > 3 IQR	max	p99
campaign	-2.0	6.0	0	3,064	6.8	1,462	63	16.0
previous	0.0	0.0	0	8,257	18.3	8,257	275	8.9

Table 9. Outlier decisions (nothing is deleted)

Variable	Decision	Reason
balance	Retain; signed-log transform (sign(x)*log(1+|x|)), then standardise	Overdrafts are valid and rich clients real; log compresses the tail while keeping the sign.
campaign, previous	Retain; cap at the TRAINING 99th percentile, then log(1+x), then standardise	Extreme counts (e.g. previous = 275) may be entry errors; capping bounds their influence without deleting rows.
pdays_clean	Retain; log(1+x), standardise	Right-skewed time since last contact.
duration	Retain for EDA; EXCLUDED from the primary model	Known only after the call (leakage); sensitivity run includes it with cap + log.
age	Retain; standardise only	1.1% flagged, all plausible ages (max 95).
day	Retain; standardise	No outliers.

3.4 Transformations, encoding and scaling (leakage-safe)

All learned preprocessing (quantile caps, means and standard deviations for scaling, one-hot category lists) is fitted on the training split only and then applied to validation and test data (scikit-learn ColumnTransformer, src/preprocessing.py). Categorical variables are one-hot encoded with unknown-category handling, producing 51 model inputs from 16 raw fields (6 numeric, 1 flag, 9 categorical expanded to 44 indicator columns).

Table 10. Before-and-after comparison of skewness and excess kurtosis on the training data

variable	skew_before	skew_after	kurtosis_before	kurtosis_after
balance	8.32	-1.58	133.40	1.79
campaign	4.91	1.18	38.89	1.14
previous	7.67	2.34	103.50	4.56
pdays_clean	2.61	1.72	6.83	1.02

Table 11. Data-quality summary before and after preprocessing

Data-quality measure	Before	After
Max |skewness| of balance / campaign / previous / pdays	8.3	2.3
Max excess kurtosis of those variables	133	4.6
Hidden-missing categories	'unknown' text labels + pdays = -1	explicit categories + flag; none imputed
Max VIF in the descriptive logistic regression	1005	5.2
Rows removed	-	0

3.5 Train / validation / test split

Table 12. Data splits

Split	Rows	Share	Positive rate (stratified, primary)	Positive rate (chronological check)
Train	31,647	70%	11.70%	5.81%
Validation	6,782	15%	11.69%	11.52%
Test	6,782	15%	11.69%	39.34%

The primary protocol is a random split stratified on the outcome (seed 42), which keeps the 11.7% prevalence identical in all three parts. Because UCI documents that the file is ordered by date, a second, chronological split (first 70% / next 15% / last 15%) is used as a robustness check. It shows a strong temporal shift: the positive rate rises from 5.8% in the training period to 39.3% in the test period (Section 6.4). The test set is used once, for final evaluation; the decision threshold and all hyper-parameters are chosen on the validation set.

3.6 The seven stages of visualization

The assignment requires 'all 7 stages of visualization'; the framework of Fry (2007) is used: acquire, parse, filter, mine, represent, refine, interact. The assignment text does not list the stages, so this standard definition is assumed.

Table 13. Seven stages and where each is applied

Stage	Purpose	Evidence in this project
1 Acquire	Obtain the data reproducibly	src/data_loader.py downloads bank-full.csv (UCI, mirror fallback), validates shape/columns and records the SHA-256 (Table 2).
2 Parse	Understand structure and types	Tables 3-6: 17 fields, 7 numeric + 9 categorical + target, memory, hidden 'unknown' and sentinel values.
3 Filter	Clean and select	Tables 7-12: cleaning rules, outlier decisions, leakage-free transforms and splits; duration excluded from the primary model.
4 Mine	Statistics and patterns	Section 4: summaries, distribution fitting, tests, correlations, Cramer's V, logistic regression (Tables 13-18).
5 Represent	Choose encodings	Figures 1-14 (bar, histogram, boxplot, PMF overlay, Q-Q, heatmaps, pairplot, violin, forest) and 15-22 (model results).
6 Refine	Improve legibility	Integer-aligned bins, log/symlog scales, CIs, n annotations, consistent colours, clipped axes disclosed; defects found in review were fixed.
7 Interact	Explore interactively	outputs/interactive/explorer.html (Plotly): dropdown rate-by-category with CIs, histogram by outcome, hover/zoom scatter.

4. Data Visualization

Fourteen exploratory figures are presented (assignment minimum: five). Each is accompanied by the question it answers, what it shows, why the chart type was chosen, the variables, the actual finding, its interpretation and the caveats. All statistics are computed from the data by the code in the repository.

4.1 Univariate analysis

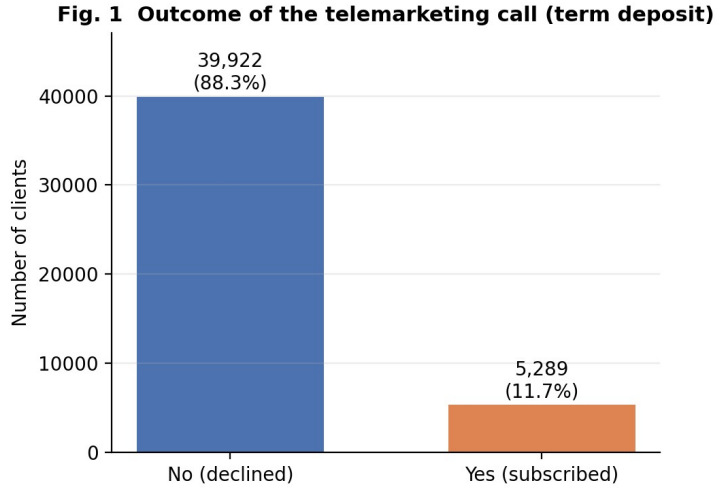


Figure 1. Class balance of the target variable.

Question	How imbalanced is the outcome (did the client subscribe to a term deposit)?
What it shows	Number and share of clients in each outcome class.
Why this chart	A bar chart is the simplest faithful encoding of a binary categorical variable and the class ratio decides which evaluation metrics are meaningful.
Variables	y (categorical, binary).
Finding	5,289 of 45,211 clients subscribed (p-hat = 11.70%, exact 95% CI 11.40%-12.00%); the classes are about 7.5:1.
Interpretation	The outcome is well modelled as a Bernoulli(p = 0.117) variable. A classifier that always answers 'no' would already be 88.3% accurate, so accuracy is uninformative and ROC-AUC, PR-AUC and F1 are used instead.
Caveats	The rate describes this particular campaign and its client selection, not the general population of bank customers.

Fig. 2 Univariate distributions of the numeric variables

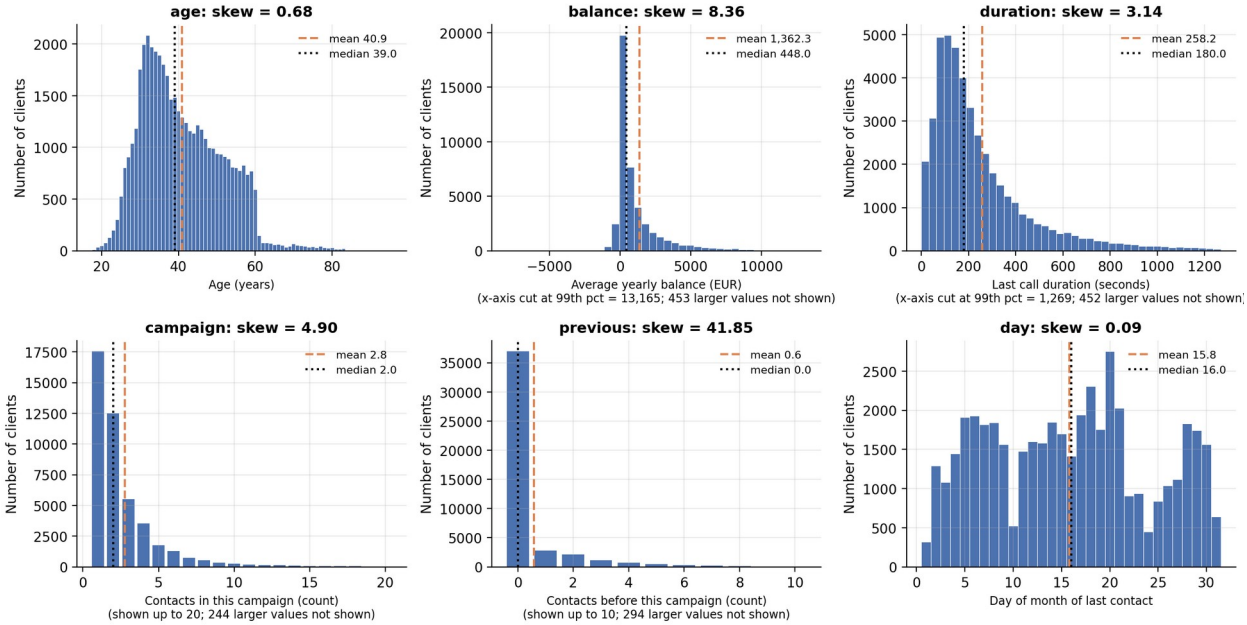


Figure 2. Univariate distributions of the numeric variables.

Question	What shape do the numeric variables have (symmetry, spread, tails, discreteness)?
What it shows	Histograms (bars for the two count variables) with the mean (dashed) and median (dotted).
Why this chart	Histograms expose skewness, modality and tail weight, which determine which transformations and robust statistics are needed.
Variables	age, balance, duration, campaign, previous, day.
Finding	age is mildly right-skewed (skew 0.68, mean 40.9 vs median 39). balance is extremely skewed (skew 8.36, excess kurtosis 141; mean EUR 1,362 vs median EUR 448). duration is right-skewed (skew 3.14; mean 258 s vs median 180 s). campaign (median 2, max 63) and previous (81.7% zeros, max 275) are zero/one-heavy counts with long tails. day is roughly flat (skew 0.09).
Interpretation	For balance, duration, campaign and previous the mean is a poor summary of a typical client; medians and quantiles are more representative, and log-type transformations are appropriate before modelling. The flat day-of-month profile suggests no strong calendar-day trend.
Caveats	The balance, duration, campaign and previous axes are truncated for readability (counts of hidden values are in the axis labels); the statistics in the panel titles use all records.

Fig. 3 Outlier screening with the IQR rule (whiskers = 1.5 x IQR)

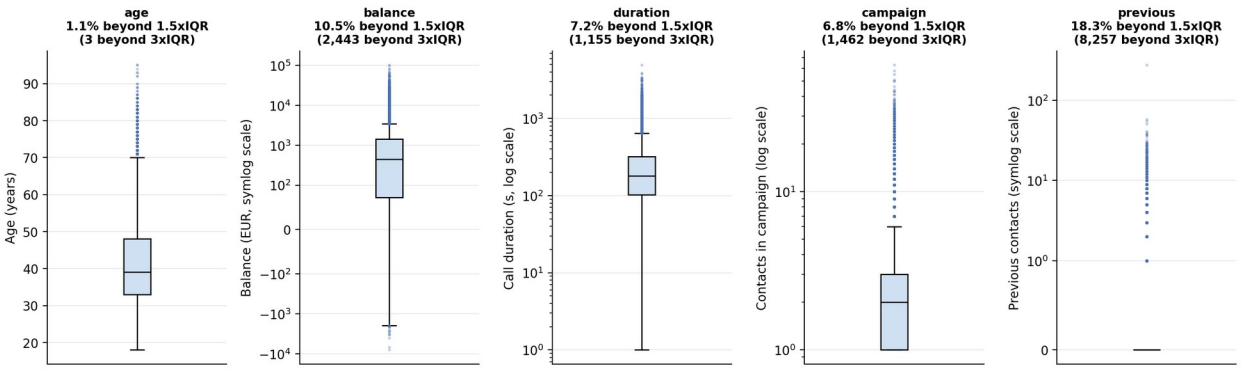


Figure 3. Outlier screening with the IQR rule.

Question	How many observations lie far outside the bulk of each numeric variable, and are they errors?

What it shows	Boxplots with 1.5 x IQR whiskers on log / symmetric-log scales; titles give the share of flagged points.
Why this chart	Boxplots are the standard Tukey tool for outliers; log scales make heavy-tailed variables readable.
Variables	age, balance, duration, campaign, previous.
Finding	Flagged by 1.5 x IQR: balance 10.5% (4,729 records), duration 7.2%, campaign 6.8%, age 1.1%, previous 18.3% (degenerate: IQR = 0). The largest values are previous = 275, campaign = 63, balance = EUR 102,127, duration = 4,918 s.
Interpretation	These are plausible behaviours (wealthy clients, long conversations, persistent re-dialling) rather than obvious errors, and subscribers cluster among long calls, so deleting them would discard signal. Decision: retain all records; compress tails with a signed-log transform (balance) and cap-then-log (campaign, previous, at the training-set 99th percentile). The single previous = 275 record is the only value that looks like a possible entry error and is neutralised by the cap.
Caveats	The IQR rule assumes roughly symmetric data; for skewed variables it flags many legitimate values. Negative balances (overdrafts) are valid and are not outliers in the cleaning sense.

Fig. 4 Category frequencies (grey bars = 'unknown', a hidden form of missing data)

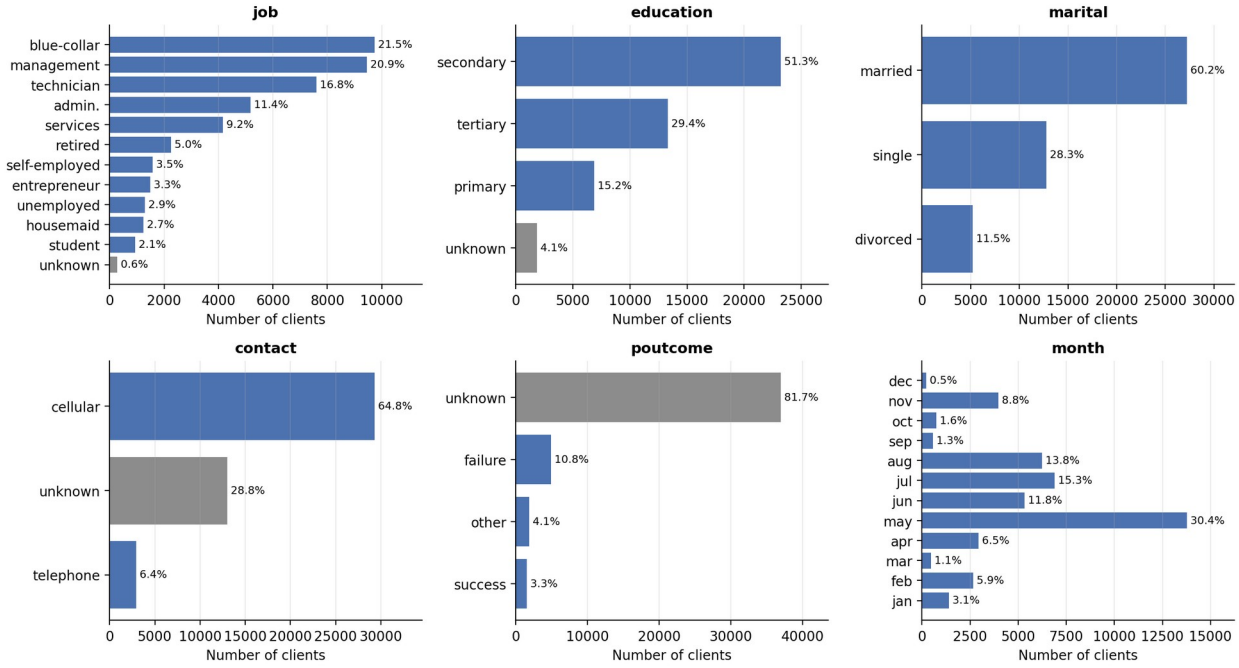


Figure 4. Category frequencies and hidden missing data.

Question	How are the categorical variables distributed, and where is information hidden behind 'unknown'?
What it shows	Horizontal bars of category shares; 'unknown' levels are grey.
Why this chart	Bar charts are the correct encoding for nominal variables; ordering by size (or calendar order for month) aids comparison.
Variables	job, education, marital, contact, poutcome, month.
Finding	The file has no NaN cells, yet 'unknown' accounts for 81.7% of poutcome, 28.8% of contact, 4.1% of education and 0.6% of job. May alone holds 30.4% of all calls; married clients are 60.2%; blue-collar and management jobs together are 42.4%.
Interpretation	poutcome = 'unknown' (36,959 clients) coincides with pdays = -1 ('never contacted before', 36,954 clients) up to 5 records, so it is structural, not random, missingness. Therefore 'unknown' is kept as its own category instead of being imputed with the mode.
Caveats	The calls are heavily concentrated in a few months, so the client mix is not representative of the whole year.

4.2 Discrete and continuous probability distributions

Table 14. Bernoulli model of the outcome

Model	Estimate	Interval	Variance	Assumption check
y ~ Bernoulli(p)	p-hat = 0.1170	exact 95% CI [0.1140, 0.1200]	variance p(1-p) = 0.1033	homogeneity across months: chi2(11) = 3062, p < 0.001 -> a single common p is NOT adequate

Table 15. Discrete distributions fitted to the count variables (max-likelihood estimates)

Variable	Mean	Variance	Var/mean	AIC Poisson	AIC NB	% zeros: obs / Poisson / NB	NB chi-square
campaign - 1	1.76	9.60	5.44	220,368	162,240	38.8 / 17.1 / 42.0	1786 (df 7)
previous	0.58	5.31	9.14	132,952	75,471	81.7 / 56.0 / 81.6	384 (df 7)

Table 16. Continuous distributions fitted to age

distribution	params	loglik	AIC	delta_AIC	KS_D	KS_p
lognormal	(np.float64(0.2546), np.float64(0.0), np.float64(39.6239))	-168,646	337,297	0	0.061	1.4e-146
gamma	(np.float64(15.5109), np.float64(0.0), np.float64(2.6392))	-169,014	338,032	735	0.073	1.4e-209
normal	(np.float64(40.9362), np.float64(10.6186))	-170,968	341,939	4,642	0.096	<1e-300

Table 17. Continuous distributions fitted to call duration (duration > 0)

distribution	params	loglik	AIC	delta_AIC	KS_D	KS_p
lognormal	(np.float64(0.9328), np.float64(0.0), np.float64(174.6112))	-294,393	588,789	0	0.038	4.2e-57
gamma	(np.float64(1.4221), np.float64(0.0), np.float64(181.5481))	-294,727	589,457	668	0.057	8.0e-127
weibull	(np.float64(1.1594), np.float64(0.0), np.float64(273.429))	-295,417	590,838	2,049	0.061	4.3e-145
exponential	(np.float64(0.0), np.float64(258.1802))	-296,278	592,558	3,768	0.106	<1e-300

Interpretation of the tables: with n of about 45,000 every KS or chi-square test rejects every candidate (p-values are essentially zero), so these tests cannot rank the models. The KS distance D (maximum gap between the empirical and fitted CDF) and the AIC differences are therefore used to judge how close each model is, and Q-Q plots to see where it fails.

Fig. 7 Discrete distributions: observed count PMF vs. fitted Poisson and Negative Binomial

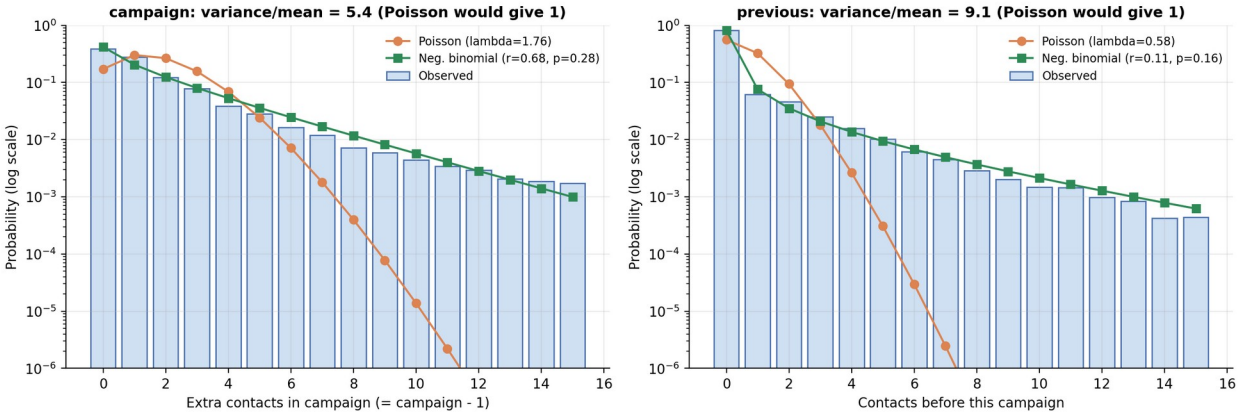


Figure 7. Discrete distributions: Poisson versus Negative Binomial.

Question	Can the number of contacts be modelled as Poisson events, or is the data over-dispersed?
What it shows	Observed probability mass function (bars) against fitted Poisson and Negative Binomial PMFs, log-scaled y-axis.
Why this chart	Overlaying fitted PMFs on a log axis is the clearest way to compare tail behaviour, which is where count models usually fail.
Variables	campaign - 1 (extra contacts in this campaign), previous.
Finding	campaign - 1: mean 1.76, variance 9.60, variance/mean = 5.4. Zeros: observed 38.8%, Poisson 17.1%, Negative Binomial 42.0%. P(>= 10 extra contacts): observed 2.65%, Poisson 0.0016%, NB 1.94%. AIC: Poisson 220,368 vs NB 162,240. previous: variance/mean = 9.1, observed zeros 81.7% vs Poisson 56.0% vs NB 81.6%.
Interpretation	The Poisson assumption (variance = mean, constant event rate) fails badly. The Negative Binomial (a Poisson-Gamma mixture, allowing client-to-client differences in contactability) captures the heavy tail and the excess zeros far better (AIC lower by 58,128). Even so, chi-square goodness-of-fit still rejects it (campaign: chi2 = 1786, df = 7, p < 0.001), so it is a reasonable approximation, not an exact model.
Caveats	With n = 45,211 any imperfect model is rejected; the relevant question is practical closeness. Contact counts are produced by bank policy (stop rules), so independence is questionable. previous is zero-inflated (a hurdle model would be a natural next step).

Fig. 8 Continuous distributions: is age normally distributed?

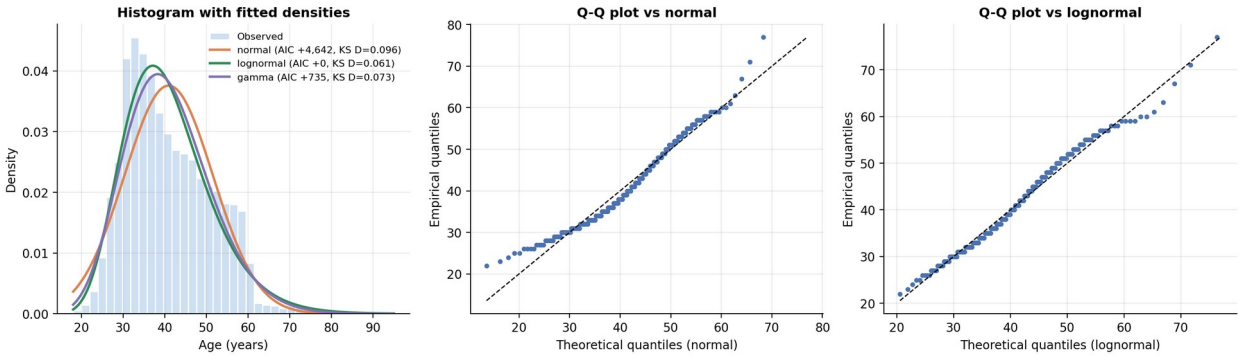


Figure 8. Continuous distributions: is age normal?.

Question	Is age approximately normally distributed, and if not, which family is closer?
What it shows	Histogram with fitted Normal, Lognormal and Gamma densities (left) and Q-Q plots against Normal and Lognormal (centre, right).
Why this chart	Density overlays show body fit; Q-Q plots reveal tail departures that overlays hide. Fit quality is judged with AIC and the KS distance, not by eye.
Variables	age.

Finding	Normality is rejected (D'Agostino K2 = 3067, p < 0.001; Shapiro-Wilk W = 0.962 on a 5,000 sample, p < 0.001); skew = 0.68. AIC ranks Lognormal best (Gamma +735, Normal +4,642); KS distance D: Normal 0.096, Gamma 0.073, Lognormal 0.061.
Interpretation	Age is right-skewed with a lower bound at 18 and a long upper tail, so the Normal is a poor description (largest CDF gap 9.6%). Lognormal is the best of the three but still leaves a maximum CDF gap of 6.1%, so no simple family is accurate and the empirical distribution should be used for inference.
Caveats	Fits fix the location at 0; a shifted distribution (e.g. starting at 18) might fit better. KS p-values with estimated parameters are only indicative, so D and AIC are reported as effect sizes.

Fig. 9 Continuous distributions: is call duration a memoryless (exponential) waiting time?

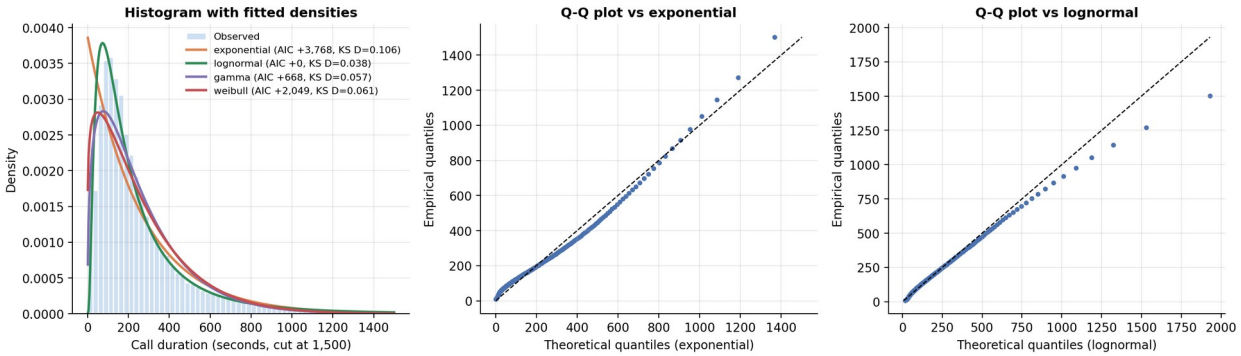


Figure 9. Continuous distributions: is call duration exponential?.

Question	Does call duration behave like a memoryless (exponential) waiting time?
What it shows	Histogram with fitted Exponential, Lognormal, Gamma and Weibull densities (left) and Q-Q plots against Exponential and Lognormal.
Why this chart	Durations are the textbook use-case for the exponential model, so the assumption is tested explicitly rather than assumed.
Variables	duration (seconds; 45,208 calls with duration > 0).
Finding	AIC ranks Lognormal best (Gamma +668, Weibull +2,049, Exponential +3,768). KS distance D: Lognormal 0.038, Gamma 0.057, Weibull 0.061, Exponential 0.106. Calls shorter than 30 s: observed 4.3% vs 11.0% expected under the fitted Exponential.
Interpretation	The exponential model overstates very short calls (its density peaks at zero) whereas real calls have a mode near 100 s: people rarely hang up instantly but converse for a typical length, so the process is not memoryless. The exponential's median (179 s) happens to match the observed median (180 s), a good reminder that matching one summary is not goodness of fit; the Q-Q plot against the Exponential looks deceptively close in the upper tail while D = 0.106 and the AIC gap show the body is wrong.
Caveats	Only the last call to each client is recorded and duration is not known before a call, so it cannot be used for pre-call prediction (see Section 7).

4.3 Bivariate and multivariate analysis

Fig. 5 Subscription rate by client / campaign category (bars = rate, whiskers = exact 95% CI)

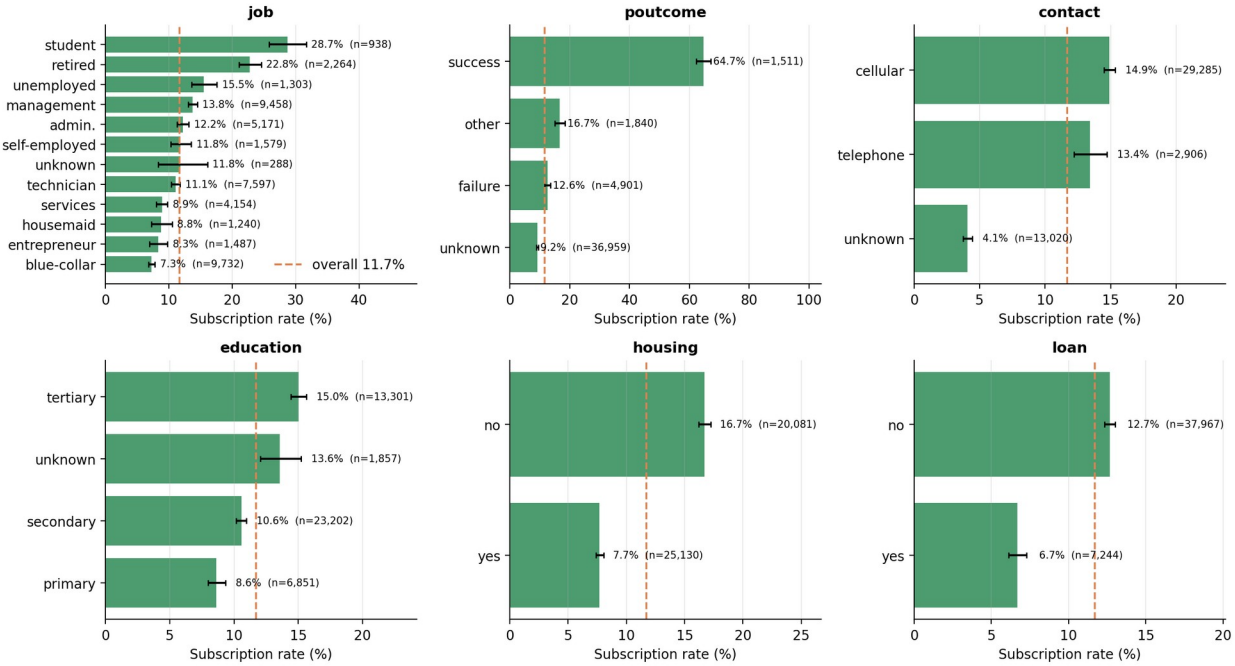


Figure 5. Subscription rate by category.

Question	Which client and campaign categories convert more or less often than average?
What it shows	Subscription rate per level with exact (Clopper-Pearson) 95% CIs, sorted; dashed line = overall rate.
Why this chart	Showing rates with confidence intervals and n prevents over-reading small groups; counts alone (Fig. 4) would not show conversion.
Variables	job, poutcome, contact, education, housing, loan vs. y.
Finding	Previous success is the standout: poutcome = success converts at 64.7% (n = 1,511) versus 9.2% for unknown. Students (28.7%) and retired clients (22.8%) exceed blue-collar (7.3%). Unknown contact type converts at 4.1% vs 14.9% (cellular). Clients without a housing loan subscribe at 16.7% vs 7.7% with one; without a personal loan 12.7% vs 6.7%.
Interpretation	Existing positive relationships and clients with fewer credit commitments are the most receptive; a bank could prioritise them. These are descriptive associations: they do not show that a loan causes refusal.
Caveats	These are marginal rates, not adjusted for each other. contact = unknown is concentrated in June (85% of June calls) and May (58%), so it partly reflects timing.

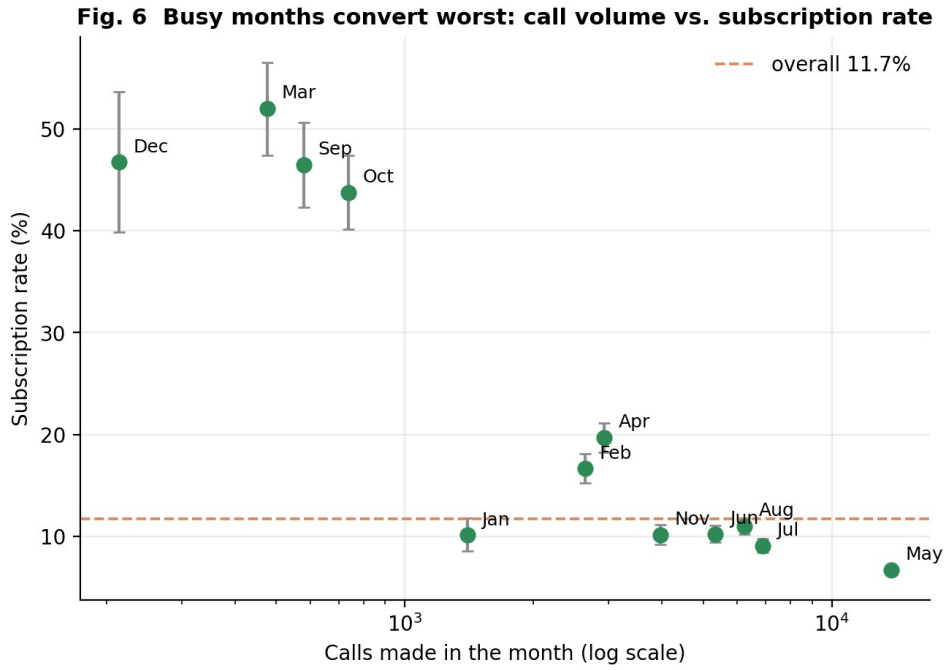


Figure 6. Call volume versus subscription rate by month.

Question	Is the success rate constant over the calendar, as a single Bernoulli/binomial model would assume?
What it shows	One point per month: calls made (log x-axis) against subscription rate with 95% CI.
Why this chart	A scatter of volume against rate exposes the relationship between effort and outcome that a plain monthly bar chart hides.
Variables	month, y.
Finding	May has 13,766 calls but the lowest rate (6.7%); Mar, Sep, Oct, Dec have only 2,008 calls in total, with rates of 44%-52%. Monthly rates range from 6.7% to 52.0%; homogeneity chi-square(11) = 3062, p < 0.001, Cramer's V = 0.26.
Interpretation	The success probability is clearly not the same in every month, so one common Bernoulli/binomial parameter is inadequate and month must be a model input. Busy months are the least productive, consistent with mass-calling broad lists versus selectively calling warmer clients.
Caveats	The data have no year field and span 2008-2010, so month mixes campaign phases. Low-volume months were probably selectively targeted (88% of May clients hold housing loans vs 18-24% in Mar/Sep/Oct/Dec), so this is not evidence that calling less causes higher conversion.

Table 18. Numeric variables by outcome: Mann-Whitney U test with Cliff's delta (effect size)

variable	median_yes	median_no	mean_yes	mean_no	p_value	cliffs_delta
duration	426.00	164.00	537.29	221.18	<0.001	0.62
previous	0.00	0.00	1.17	0.50	<0.001	0.20
pdays_clean	0.00	0.00	69.34	37.26	<0.001	0.19
balance	733.00	417.00	1,804.27	1,303.71	<0.001	0.18
campaign	2.00	2.00	2.14	2.85	<0.001	-0.14
day	15.00	16.00	15.16	15.89	<0.001	-0.05
age	38.00	39.00	41.67	40.84	0.063	-0.02

Table 19. Categorical variables versus outcome: chi-square test and bias-corrected Cramer's V

variable	chi2	dof	p_value	cramers_v
poutcome	4,391.507	3	<0.001	0.312
month	3,061.839	11	<0.001	0.260
contact	1,035.714	2	<0.001	0.151
housing	874.822	1	<0.001	0.139

variable	chi2	dof	p_value	cramers_v
job	836.105	11	<0.001	0.135
education	238.924	3	<0.001	0.072
loan	209.617	1	<0.001	0.068
marital	196.496	2	<0.001	0.066
default	22.202	1	<0.001	0.022

Group comparisons beyond two groups: balance differs across job categories (Kruskal-Wallis H = 565, p < 0.001, epsilon-squared = 0.012, a small effect) and age differs across education levels (H = 1955, p < 0.001, epsilon-squared = 0.043, small). Non-parametric tests are used because balance and duration are heavy-tailed; all p-values should be read together with effect sizes.

Fig. 10 Correlation structure of the numeric variables and the outcome

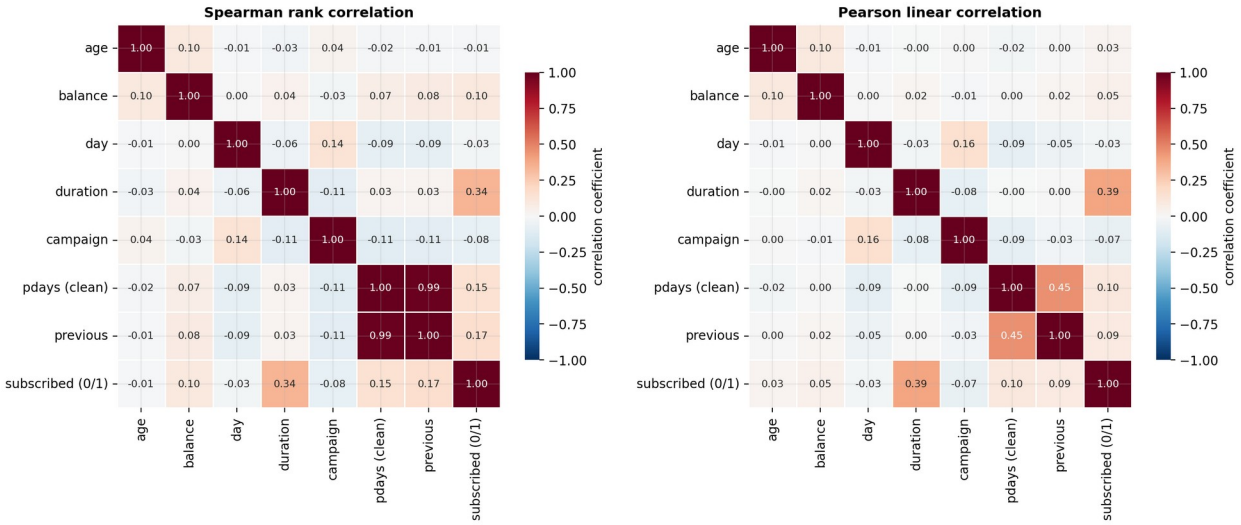


Figure 10. Correlation heatmaps (Spearman and Pearson).

Question	Which numeric variables move together, and which relate to the outcome?
What it shows	Annotated correlation matrices; Spearman (rank-based, monotonic) beside Pearson (linear).
Why this chart	The two measures are shown together because the variables are heavily skewed: Spearman is robust to the tails, and any gap between the two reveals outlier influence.
Variables	age, balance, day, duration, campaign, pdays (clean), previous, subscribed (0/1).
Finding	With the outcome: duration rho = 0.34 (Pearson 0.39), previous 0.17, pdays 0.15, balance 0.10, campaign -0.08, age -0.01. Among predictors, only previous-pdays is strong (rho = 0.99); the next largest |rho| is campaign-day = 0.14.
Interpretation	All single-variable associations with the outcome are weak to moderate (|rho| <= 0.34); duration is strongest but unavailable before the call. previous and pdays are nearly redundant, which is why one of them is dropped from the logistic regression (VIF 1005 -> 5.2). balance has Pearson 0.05 but Spearman 0.10: extreme values weaken the linear measure.
Caveats	Correlation is not causation, and a coefficient near zero does not rule out a non-monotonic relationship: age has rho = -0.01 yet a strong U-shaped pattern (Fig. 13).

Fig. 11 Association between categorical variables (and the outcome 'y')

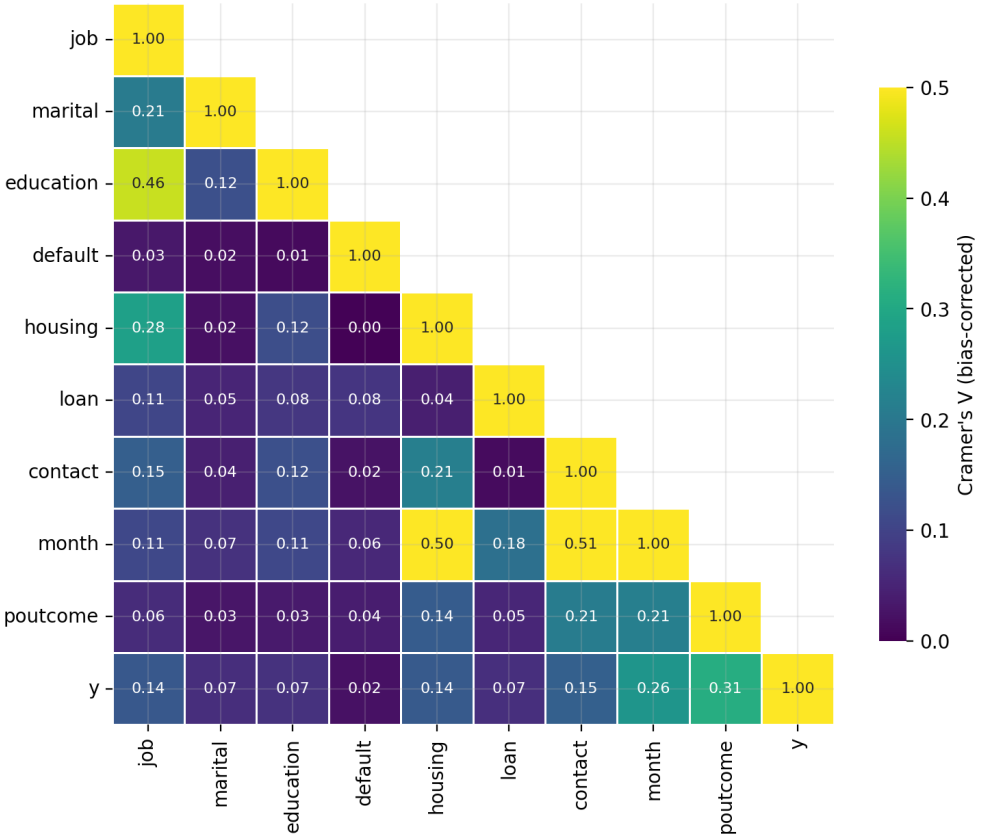


Figure 11. Association between categorical variables (Cramer's V).

Question	Which categorical variables are associated with the outcome and with each other?
What it shows	Lower-triangle heatmap of bias-corrected Cramer's V (0 = independent, 1 = perfectly associated).
Why this chart	Pearson/Spearman do not apply to nominal variables; Cramer's V (from chi-square) puts all pairs on one 0-1 scale.
Variables	job, marital, education, default, housing, loan, contact, month, poutcome, y.
Finding	Strongest links to y: poutcome 0.31, month 0.26, contact 0.15, housing 0.14, job 0.14; default only 0.02. Among predictors: month-contact 0.51, month-housing 0.50, job-education 0.46.
Interpretation	Previous campaign outcome and timing are the most informative categorical signals, while having credit in default carries almost no information. The large month-contact and month-housing values show that the client mix and the contact channel changed over the campaign, so marginal differences (Fig. 5) partly reflect timing.
Caveats	Cramer's V is symmetric and says nothing about direction; every chi-square test here is 'significant' because n is large, so effect size (V) matters more than p.

Fig. 12 Pairplot of transformed numeric variables (random sample of 2,000 clients)

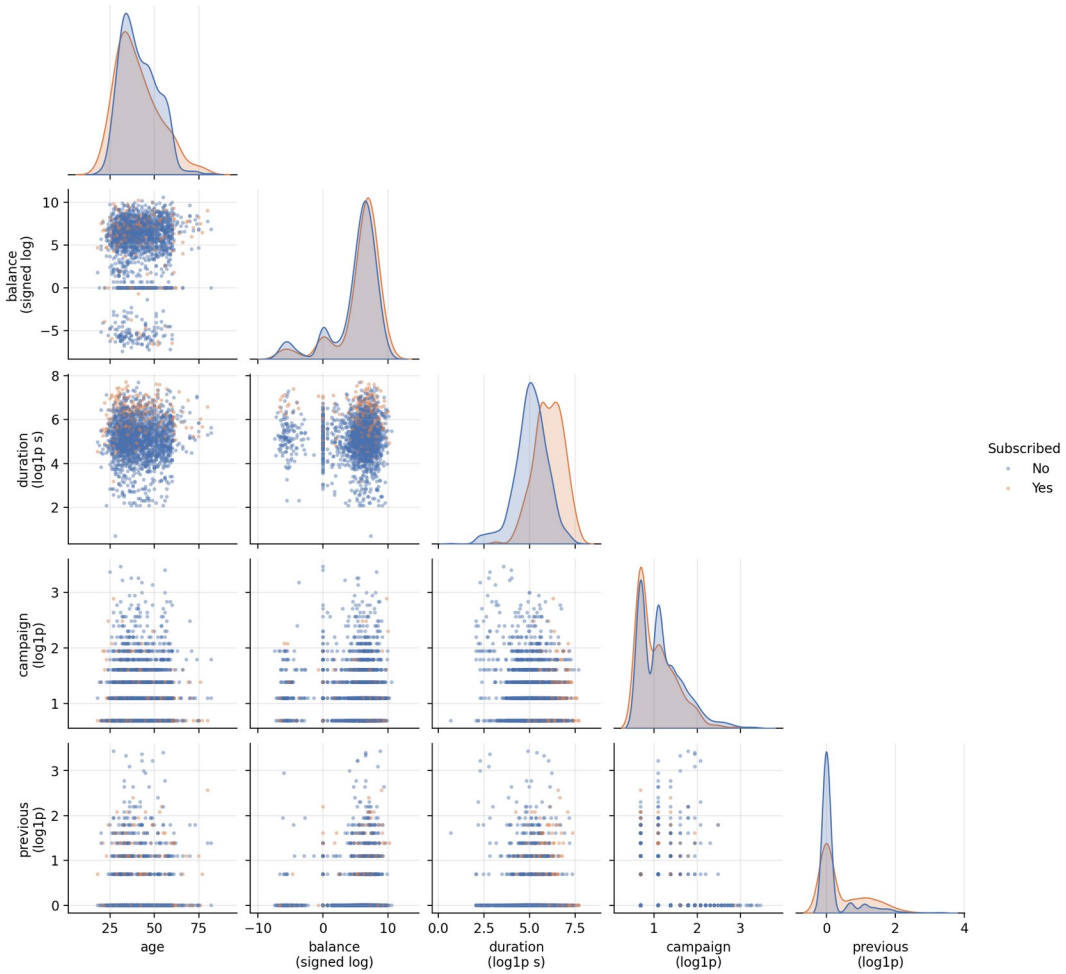


Figure 12. Pairplot of transformed numeric variables.

Question	Do pairs of numeric variables show joint structure, clusters or separation by outcome?
What it shows	Scatter matrix (lower triangle) and class-wise KDEs (diagonal) of 2,000 random clients on log-type scales.
Why this chart	A pairplot gives a one-glance view of all bivariate relationships and class overlap; a sample keeps it legible.
Variables	age, balance (signed log), duration (log1p), campaign (log1p), previous (log1p), coloured by y.
Finding	balance on the signed-log scale is clearly multi-modal: a cluster of overdrawn accounts, a spike at exactly zero (3,514 clients = 7.8%) and a main mass of positive balances. Subscribers (orange) are shifted towards longer durations, but no pair of predictors shows a strong linear relation and the classes overlap heavily everywhere else.
Interpretation	No two predictors are strongly entangled (consistent with Fig. 10), and no simple two-variable boundary separates subscribers, which anticipates the moderate predictive ceiling seen later. The multi-modality of balance is invisible in the raw histogram.
Caveats	Based on a random sample of 2,000 rows (seed 42); campaign/previous form discrete lattices; overplotting hides density in the centre.

Fig. 13 Age and account balance versus subscription

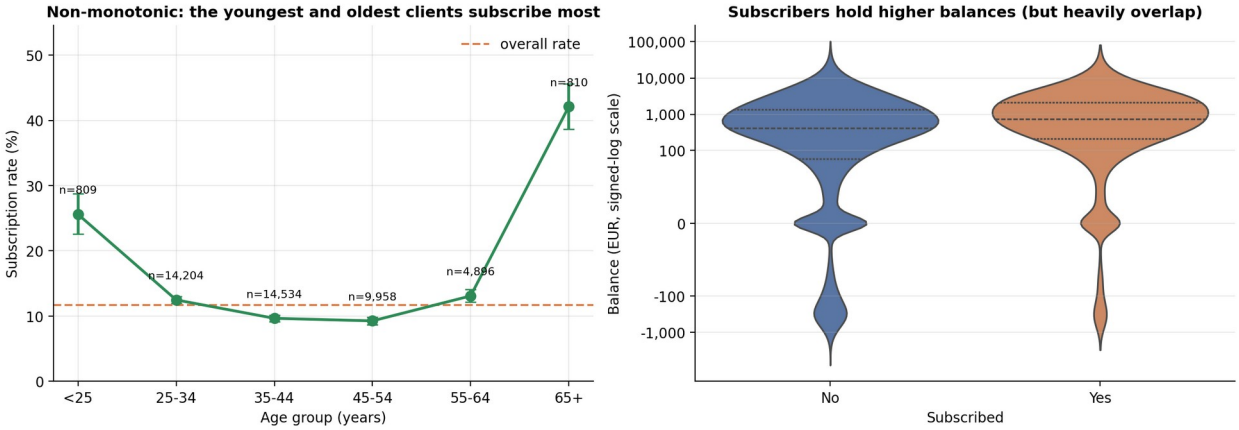


Figure 13. Age and balance versus subscription.

Question	Does the relation between age or balance and subscription follow a simple monotonic trend?
What it shows	Left: subscription rate per age band with exact 95% CIs. Right: violin plots of signed-log balance by outcome with quartiles.
Why this chart	Binning age reveals shape that a single correlation coefficient hides; violins show full distributions and the overlap between classes.
Variables	age (grouped), balance, y.
Finding	Rate by age: <25 25.6% (n = 809), 45-54 9.3%, 65+ 42.1% (n = 810); yet Spearman rho(age, y) = -0.01 and Mann-Whitney p = 0.063 (median age 38 vs 39). Balance median EUR 733 (subscribers) vs EUR 417; Mann-Whitney p < 0.001, Cliff's delta = 0.18.
Interpretation	Age matters, but not linearly: the youngest and oldest clients are far more receptive than mid-career clients, which monotone tests (correlation, Mann-Whitney) miss completely. Higher balance is associated with subscription, but the small effect size (delta = 0.18) and the heavy overlap in the violins mean balance alone is a weak discriminator.
Caveats	Groups <25 and 65+ are small (about 800 each) and overlap with students and retirees (Fig. 5), so age and occupation effects cannot be separated here.

Fig. 14 Multivariable logistic regression: the 16 strongest adjusted associations

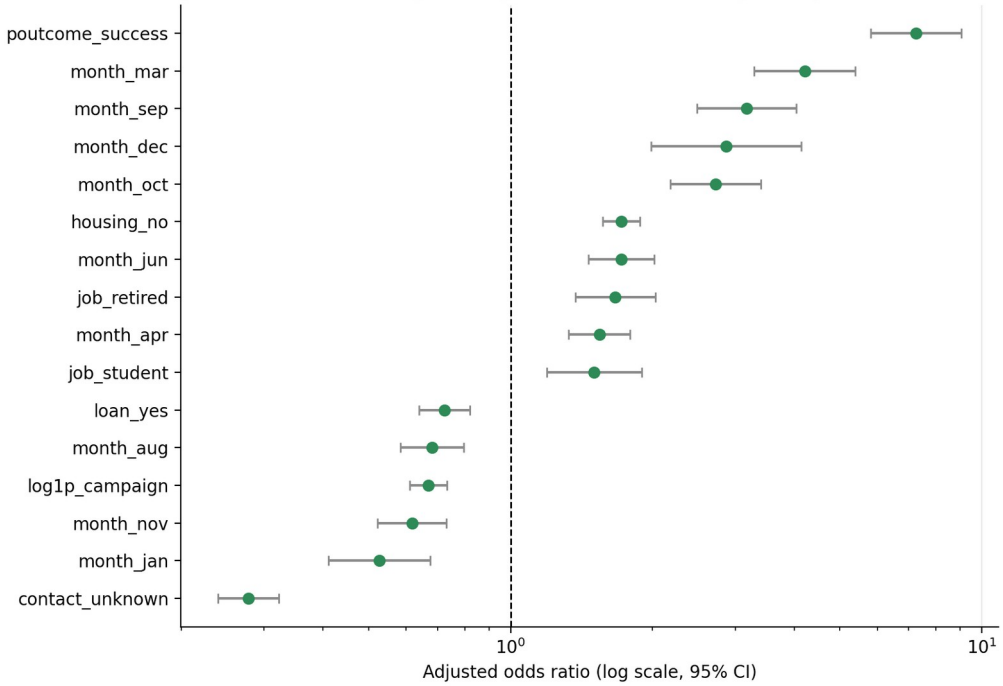


Figure 14. Adjusted odds ratios from a multivariable logistic regression.

Question	Which factors remain associated with subscription after adjusting for all the others?
What it shows	Odds ratios (log scale) with 95% CIs for the 16 largest effects; the dashed line at 1 means no association.
Why this chart	A forest plot is the standard way to compare adjusted effects on a common multiplicative scale with their uncertainty.
Variables	All pre-call predictors (n = 31,647 training clients; McFadden pseudo-R2 = 0.173).
Finding	poutcome = success: OR 7.24 (95% CI 5.80-9.04); March OR 4.21; contact unknown OR 0.28 (0.24-0.32); no housing loan OR 1.72; each unit of log(1 + campaign) OR 0.67. Before removing redundant predictors the maximum VIF was 1005 (5 variables above 10); afterwards 5.2.
Interpretation	After adjustment, a past success, particular months, a known contact channel and the absence of a housing loan are still associated with higher odds, and repeated contacting with lower odds. The model explains a modest share of variation (pseudo-R2 about 0.17), in line with the weak individual associations seen above.
Caveats	Odds ratios describe association, not causation. Month effects absorb campaign phase; the first fit with all history variables had unstable intervals because previous, pdays and previously_contacted are redundant, which is why only log(1 + previous) was retained.

Table 20. Logistic regression (training set, n = 31,647, pseudo-R2 = 0.173): 14 largest adjusted odds ratios

term	odds_ratio	ci_low	ci_high	p_value
poutcome_success	7.243	5.803	9.041	<0.001
month_mar	4.208	3.283	5.394	<0.001
contact_unknown	0.278	0.239	0.322	<0.001
month_sep	3.169	2.488	4.036	<0.001
month_dec	2.870	1.988	4.143	<0.001
month_oct	2.725	2.183	3.400	<0.001
month_jan	0.526	0.411	0.675	<0.001
housing_no	1.720	1.571	1.883	<0.001
month_jun	1.717	1.461	2.019	<0.001
job_retired	1.668	1.371	2.029	<0.001
month_nov	0.618	0.523	0.730	<0.001
month_apr	1.543	1.327	1.794	<0.001
job_student	1.506	1.194	1.898	<0.001

term	odds_ratio	ci_low	ci_high	p_value
log1p_campaign	0.669	0.612	0.732	<0.001

4.4 Key statistical findings

Table 21. Evidence-based summary of the main findings

#	Observation	Statistic (from the data)	Chart	Real-world implication	Caveat
1	Only about one client in nine subscribes.	5,289/45,211 = 11.70% (95% CI 11.40%-12.00%)	Fig. 1	Accuracy is uninformative (always answering 'no' gives 88.3%); the model is judged by ROC-AUC, PR-AUC and F1.	Rate is specific to this campaign's client selection.
2	Missing data hide behind 'unknown' labels and a -1 sentinel, not NaN.	NaN cells = 0, but poutcome unknown 81.7%, contact unknown 28.8%, pdays = -1 for 81.7%	Fig. 4	'unknown' is kept as a category and pdays is split into a flag plus a cleaned value; imputing the mode would erase real information.	contact = unknown is concentrated in May-June, so it mixes channel and timing.
3	Balance, duration and the count variables are heavy-tailed; age is not normal.	balance skew 8.36, excess kurtosis 141; age Normal KS D = 0.096 (AIC +4,642 vs Lognormal)	Figs. 2, 3, 8	Signed-log and cap-then-log transforms are used; medians and rank-based tests replace means and t-tests.	No simple parametric family fits age well (best KS D = 0.061).
4	Contact counts are over-dispersed, so Poisson is rejected in favour of Negative Binomial.	variance/mean = 5.4; AIC 220,368 (Poisson) vs 162,240 (NB); P(>=10 extra contacts) observed 2.65% vs Poisson 0.0016%	Fig. 7	Client contactability is heterogeneous, and rare, very persistent call sequences are far more common than a Poisson model predicts.	NB still fails a chi-square test in the tail; counts are shaped by bank stop-rules.
5	Call duration is lognormal-like, not memoryless-exponential.	KS D: Lognormal 0.038 vs Exponential 0.106; AIC gap 3,768	Fig. 9	Very short calls are rare and a typical call length exists, so time already spent on a call does carry information.	Only the last call per client is recorded.
6	Conversion varies strongly by calendar month and is inversely related to call volume.	May: 13,766 calls at 6.7%; March: 477 calls at 52.0%; chi2(11) = 3062, V = 0.26	Figs. 6, 11	One Bernoulli parameter does not describe all months; campaign timing/targeting must be modelled.	No year field; low-volume months were probably selectively targeted, so this is not causal.
7	A previous successful campaign is the strongest single signal.	64.7% vs 9.2% (unknown); adjusted OR 7.24 (95% CI 5.80-9.04)	Figs. 5, 14	Re-contacting previously converted clients is likely the most productive use of calls.	Only 3.3% of clients have a previous success.
8	Age has a U-shaped, non-monotonic relation with subscription that correlation misses.	<25: 25.6%, 45-54: 9.3%, 65+: 42.1%; Spearman rho = -0.01	Fig. 13	Age should enter models as a non-linear effect; segment-specific offers for students and retirees are plausible.	Extreme groups are small (~800) and overlap with job categories.
9	Existing loan commitments go with lower subscription.	housing loan: 7.7% vs no housing loan 16.7%; adjusted OR (no housing) = 1.72	Figs. 5, 14	Clients with fewer credit commitments are a more receptive segment.	Association only; housing loans are also tied to month (V = 0.50).
10	The MLP has moderate, honest predictive power; call duration would be a leak.	test ROC-AUC 0.784 +/- 0.002 (5 seeds), PR-AUC 0.415; LR 0.759; GB 0.801; with duration 0.918; chronological 0.642	Figs. 15-22	The model can rank prospects usefully (precision about 3.7x prevalence at the F1 threshold) but is not accurate enough to automate decisions.	Random-split score is optimistic under temporal shift.

Association versus causation: all relationships above are observational. For example, the higher subscription rate of retired clients or of March calls does not show that these characteristics cause subscription; month and contact type, for instance, are strongly entangled (Cramer's V about 0.5), and campaign targeting changed over time.

5. Deep Learning Model

5.1 Problem formulation

Task: binary classification of whether a client subscribes (y = yes). Inputs: the 16 raw fields except duration (and the derived flag), expanded to 51 numeric inputs after preprocessing. Duration is excluded from the primary model because it is the length of the very call whose outcome is being predicted; it is used only in a leakage experiment (Section 6.4). Because the data are tabular with no spatial or sequential structure, a Multi-Layer Perceptron (MLP) is the appropriate deep learning architecture; CNNs, RNNs and Transformers would not exploit any structure present in these data.

5.2 Architecture and training parameters

Table 22. Selected architecture (1,697 trainable parameters)

Layer	Output shape	Configuration	Parameters
Input	(51)	-	0
Dense 1	(32)	ReLU, He-normal init, L2 = 1e-4	1664
Dropout 1	(32)	rate 0.2	0
Dense output	(1)	Sigmoid = P(subscribe)	33

Table 23. Training configuration and reproducibility

Parameter	Value
Loss	Binary cross-entropy
Optimizer / learning rate	Adam / 0.001
Batch size / maximum epochs	256 / 40
Early stopping	monitor validation ROC-AUC, patience 6, restore best weights
Regularisation	dropout 0.2, L2 0.0001
Class weighting	None
Validation strategy	Fixed stratified validation split (15%); test split untouched until final evaluation
Decision threshold	0.202 (maximises F1 on validation)
Seeds	primary seed 42; final configuration repeated with seeds [42, 43, 44, 45, 46]; Python/NumPy/TensorFlow seeded, op determinism enabled
Environment	Python 3.12.3, TensorFlow 2.21.0, 1 CPU core(s), GPU: none
Run time	full experiment 89 s on the final (cached) pass; search about 6 min

5.3 Hyper-parameter tuning

Table 24. Random search over 12 of 24 grid configurations (units x dropout x learning rate x class weight), ranked by validation PR-AUC

config_id	units	dropout	lr	class_weight	epochs_run	best_epoch	val_roc_auc	val_pr_auc
2	(32,)	0.2000	0.0010	none	40	38	0.7979	0.4555
0	(128, 64, 32)	0.4000	0.0010	none	40	36	0.8027	0.4541
6	(32,)	0.4000	0.0010	none	40	40	0.7926	0.4532
11	(128, 64, 32)	0.4000	0.0010	balanced	36	30	0.8004	0.4520
9	(64, 32)	0.2000	0.0010	balanced	30	24	0.7984	0.4517
5	(128, 64, 32)	0.2000	0.0003	balanced	40	40	0.7975	0.4507
3	(64, 32)	0.2000	0.0010	none	40	40	0.7981	0.4502
7	(128, 64,	0.4000	0.0003	none	40	39	0.7946	0.4447

config_id	units	dropout	lr	class_weight	epochs_run	best_epoch	val_roc_auc	val_pr_auc
	32)							
1	(32,)	0.2000	0.0003	balanced	40	40	0.7942	0.4398
8	(32,)	0.2000	0.0003	none	40	40	0.7888	0.4368
4	(32,)	0.4000	0.0003	balanced	40	40	0.7913	0.4363
10	(32,)	0.4000	0.0003	none	40	40	0.7850	0.4317

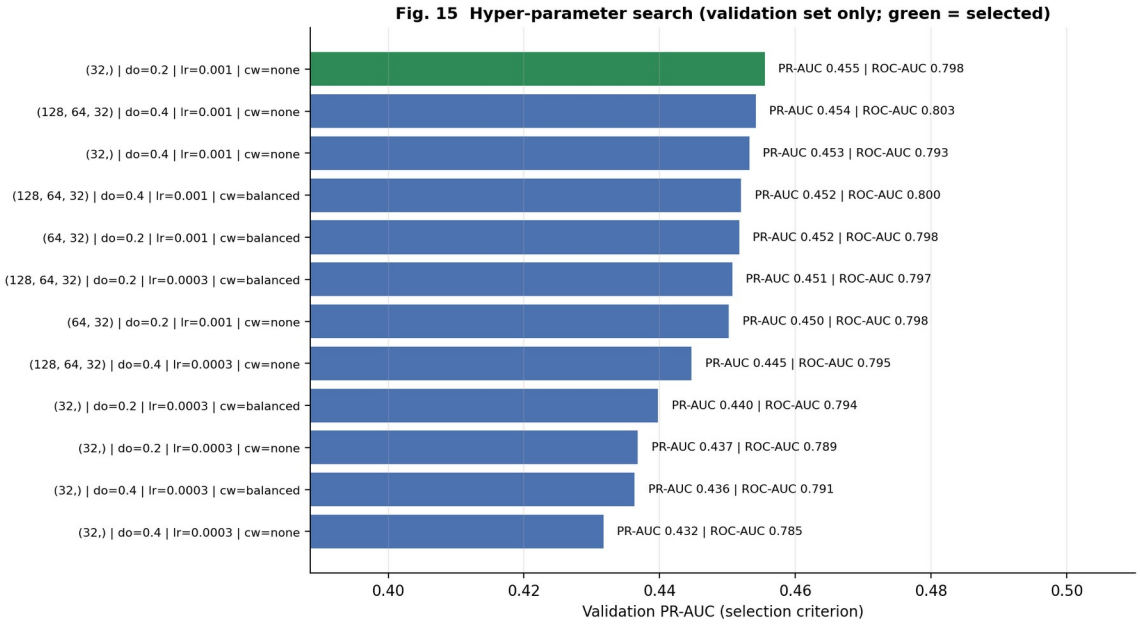


Figure 15. Hyper-parameter search.

Question	Which MLP configuration generalises best to unseen (validation) data?
What it shows	Validation PR-AUC of 12 randomly sampled configurations (units, dropout, learning rate, class weighting); green = selected.
Why this chart	A sorted bar chart shows both the spread between configurations and how much tuning actually matters.
Variables	Hidden units, dropout, learning rate, class weights.
Finding	Validation PR-AUC ranged from 0.432 to 0.455 (ROC-AUC 0.785-0.803). Selected: units (32,), dropout 0.2, lr 0.001, class weight none.
Interpretation	The spread between best and worst configuration is only 0.024 PR-AUC, so tuning matters little here; the simplest architecture won, and the three-layer networks did not outperform it. Selection used the validation set only.
Caveats	Only 12 of 24 grid points were sampled, each with one seed, so small differences are not statistically meaningful. The test set was never used for selection.

5.4 Baselines and feature analysis

Baselines are fitted on the same preprocessed data: an always-'no' (majority) classifier, logistic regression, and a histogram gradient-boosting classifier as a reference. Feature analysis combines the statistical EDA of Section 4 (Figs. 5, 10, 11, 14) with permutation importance of the trained network (Figure 20, Section 6.3).

6. Results Visualization and Interpretation

6.1 Training behaviour

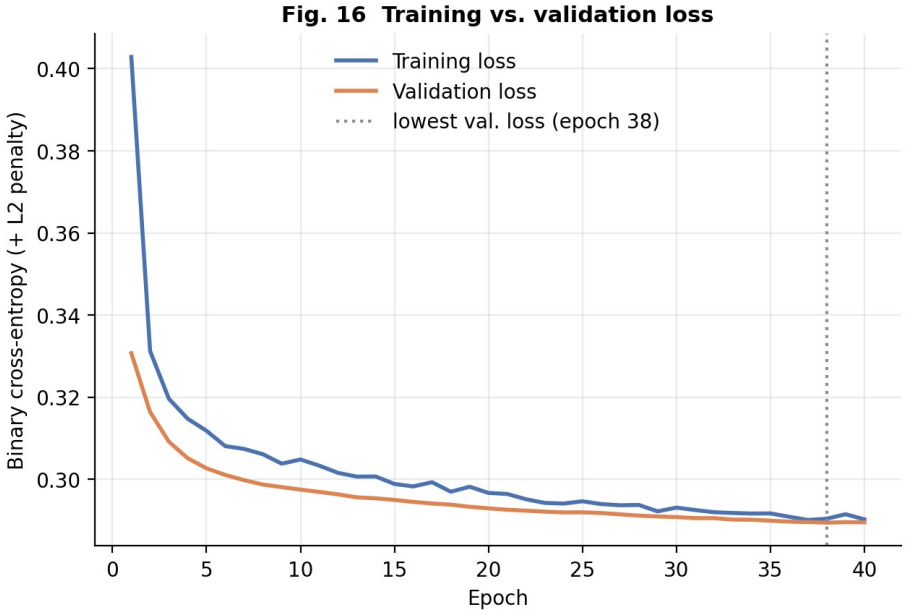


Figure 16. Training versus validation loss.

Question	Is the network underfitting or overfitting?
What it shows	Binary cross-entropy per epoch on the training and validation sets for the primary model.
Why this chart	The gap between the two loss curves is the standard diagnostic of generalisation.
Variables	Epoch, loss.
Finding	The run used all 40 allowed epochs (early stopping with patience 6 never triggered); lowest validation loss 0.289 at epoch 38 and best validation ROC-AUC at epoch 38. Final training loss 0.290 vs validation loss 0.290; first-epoch training loss 0.403.
Interpretation	Both curves fall together and end almost on top of each other, so there is no sign of overfitting (training loss is slightly higher because dropout is active during training). The curves are nearly flat by the end, so extra epochs would add very little: performance is limited by the information in the 16 pre-call features and the small network, not by memorisation.
Caveats	Training reached the 40-epoch cap while still improving marginally in the third decimal, so a longer run might give a tiny gain. Loss includes the L2 penalty.

Fig. 17 Accuracy and AUC versus epoch (accuracy alone is misleading at 11.7% prevalence)

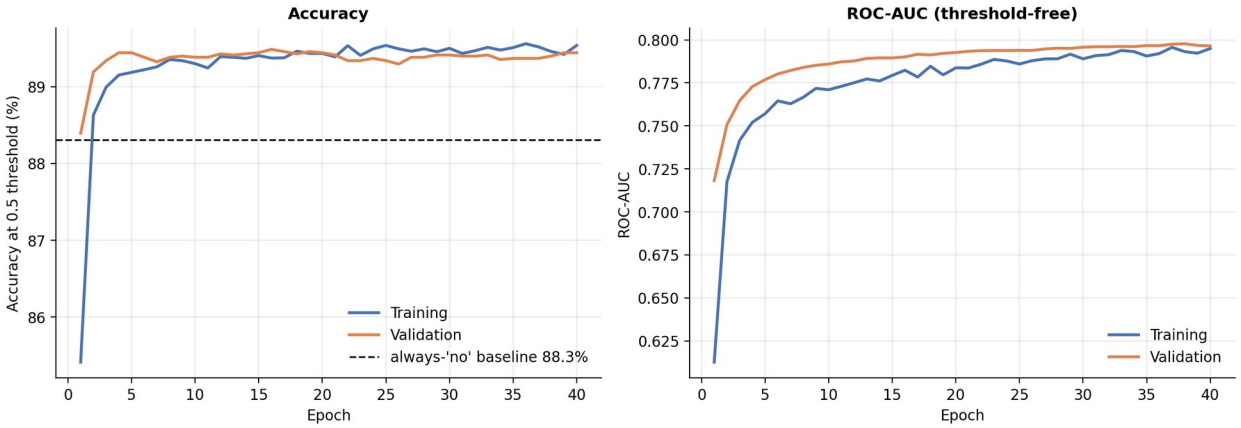


Figure 17. Accuracy and AUC versus epoch.

Question	How do accuracy and ranking quality evolve during training?
What it shows	Left: accuracy at a 0.5 threshold against the always-'no' baseline. Right: ROC-AUC per epoch (training and validation).
Why this chart	The assignment requires accuracy-versus-epoch; the AUC panel and baseline line prevent the misleading impression that high accuracy means skill when 88% of clients decline.
Variables	Epoch, accuracy, ROC-AUC.
Finding	Validation accuracy at the 0.5 threshold ends at 89.4%, only about 1.1 points above the always-'no' baseline of 88.3%; validation AUC peaks at 0.798 (epoch 38) and training AUC at 0.796.
Interpretation	Accuracy barely moves because the class prior dominates it; AUC shows the real learning (ranking ability) and the two AUC curves track each other closely, confirming the absence of overfitting seen in Fig. 16.
Caveats	Accuracy depends on the 0.5 threshold, which is not the threshold used for final reporting (Fig. 21).

6.2 Test-set performance

Table 25. Final TEST-set metrics (n = 6,782; threshold chosen on validation; prevalence 11.7%). Majority-class metrics are degenerate by construction.

Model	ROC-AUC	PR-AUC	F1	Precision	Recall	Accuracy	Bal. acc.	MCC
MLP (primary, seed 42)	0.784	0.413	0.457	0.435	0.482	0.866	0.699	0.381
MLP, mean +/- SD of 5 seeds	0.784 +/- 0.002	0.415 +/- 0.004	0.461 +/- 0.004	0.440 +/- 0.016	0.484 +/- 0.015	0.867 +/- 0.005	0.701 +/- 0.004	0.386 +/- 0.006
Majority class	0.500	0.117	0.209	0.117	1.000	0.117	0.500	0.000
Logistic regression	0.759	0.384	0.431	0.464	0.402	0.876	0.670	0.363
Gradient boosting (reference)	0.801	0.439	0.476	0.426	0.538	0.861	0.721	0.400

Reading the table: the primary MLP ranks clients with ROC-AUC 0.784; at the F1-optimal threshold it finds 48% of subscribers with 43% precision. Accuracy (86.6%) is only marginally above the 88.3% obtained by always predicting 'no', which is why it is not used as the headline metric. Seed-to-seed variation is small (SD of ROC-AUC 0.0019), so the ranking MLP > logistic regression (+0.025) is stable, while gradient boosting is better still (+0.016). The Brier score of the MLP is 0.084.

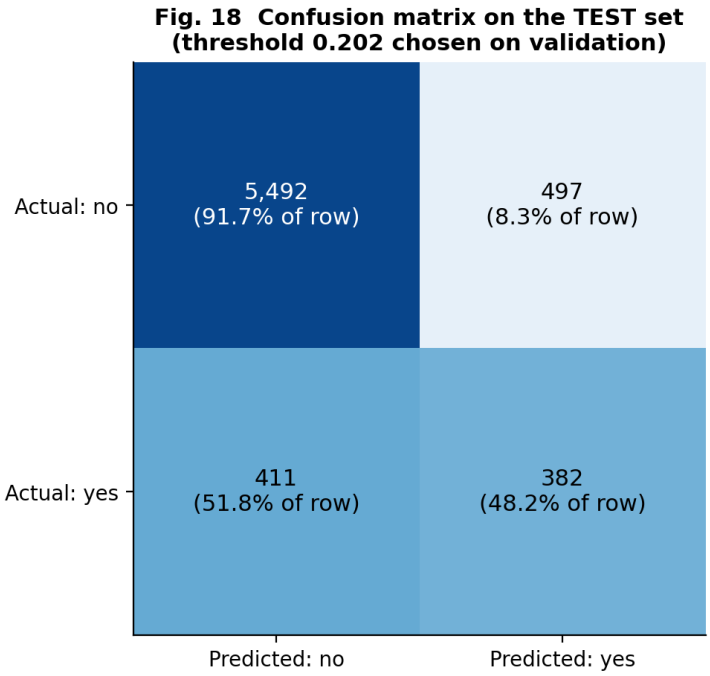


Figure 18. Confusion matrix (test set).

Question	What kinds of mistakes does the final model make at the validation-chosen threshold?
What it shows	Counts and row-percentages of true/false positives and negatives on the untouched test set.
Why this chart	A confusion matrix converts a score into concrete outcomes (missed subscribers vs wasted calls).
Variables	Predicted vs actual y; threshold 0.202 (maximises F1 on validation).
Finding	TN 5,492, FP 497, FN 411, TP 382. Recall 48.2%, precision 43.5%, F1 0.457, accuracy 86.6%, balanced accuracy 69.9%, MCC 0.381.
Interpretation	At this operating point the model finds 48% of actual subscribers but only 43% of the clients it flags actually subscribe (prevalence 11.7%), i.e. roughly 3.7x better than random targeting. The two error types are of similar size (497 wasted calls vs 411 missed subscribers), and the model still misses about half of the subscribers, so it is a prioritisation aid rather than a reliable classifier.
Caveats	The threshold optimises F1, which weights precision and recall equally; a real campaign would choose it from call cost and expected revenue.

Fig. 19 ROC and precision-recall curves on the TEST set

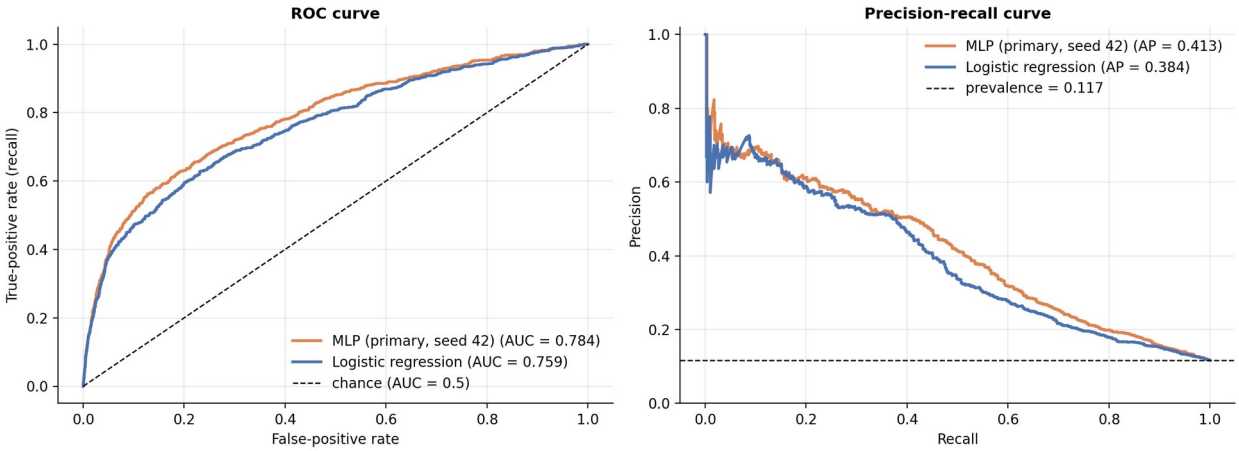


Figure 19. ROC and precision-recall curves (test set).

Question	How well does the model rank subscribers above non-subscribers across all thresholds?
What it shows	ROC curve (TPR vs FPR) and precision-recall curve for the MLP and the logistic-regression baseline.
Why this chart	ROC-AUC is threshold-free; the PR curve is more informative with 11.7% prevalence (Saito & Rehmsmeier, 2015).
Variables	Predicted probability, y.
Finding	Primary MLP: ROC-AUC 0.784, PR-AUC (average precision) 0.413 versus prevalence 0.117. Over 5 seeds ROC-AUC = 0.784 +/- 0.002. Baselines (test ROC-AUC): logistic regression 0.759, gradient boosting 0.801, majority class 0.500.
Interpretation	The MLP has real discriminative power (a random client pair is ranked correctly 78.4% of the time) and PR-AUC is about 3.5x the prevalence. The MLP beats the logistic-regression baseline by 0.025 AUC, which suggests it captures some non-linear structure, but the gradient-boosting reference is better still (+0.016), consistent with published evidence that tree ensembles are hard to beat on tabular data (Grinsztajn et al., 2022).
Caveats	Single test set of 6,782 clients: differences of about 0.01 AUC between models are within the seed-to-seed and sampling noise.

6.3 Feature importance and threshold choice

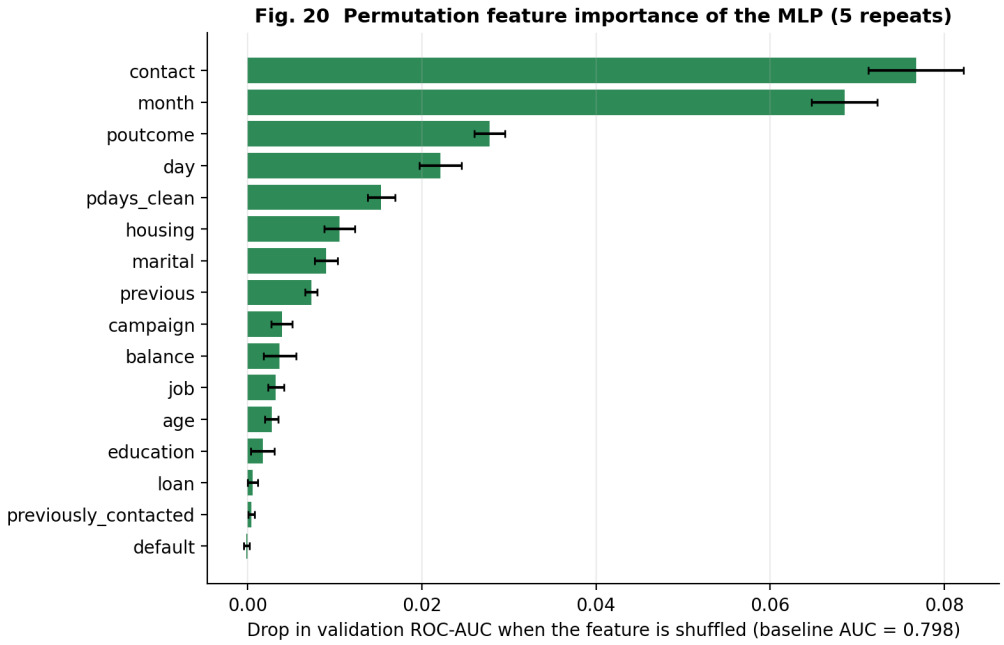


Figure 20. Permutation feature importance.

Question	Which input variables does the trained network actually rely on?
What it shows	Drop in validation ROC-AUC when each original input column is randomly shuffled (one-hot groups shuffled together, 5 repeats).
Why this chart	Permutation importance is model-agnostic and respects grouped one-hot features; it is computed on validation data to keep the test set pristine.
Variables	The 16 pre-call input columns.
Finding	Top features: contact (-0.077 AUC), month (-0.069), poutcome (-0.028); least important: default (-0.0001).
Interpretation	The ranking largely agrees with the statistical EDA: contact channel, month and previous outcome (poutcome) were among the strongest associations in Figs. 5, 6 and 11. Two points are less expected: day of month ranks fourth although its marginal correlation with the outcome is tiny (rho = -0.03), so the network probably uses it in interaction with month; and housing, the fourth-ranked categorical association with y in Fig. 11, is only mid-table because it overlaps with month and contact.
Caveats	Importance is not causation, and correlated features (month, contact, housing, day) share credit, so individual scores understate joint importance. contact and month together largely encode when in the campaign the call happened.

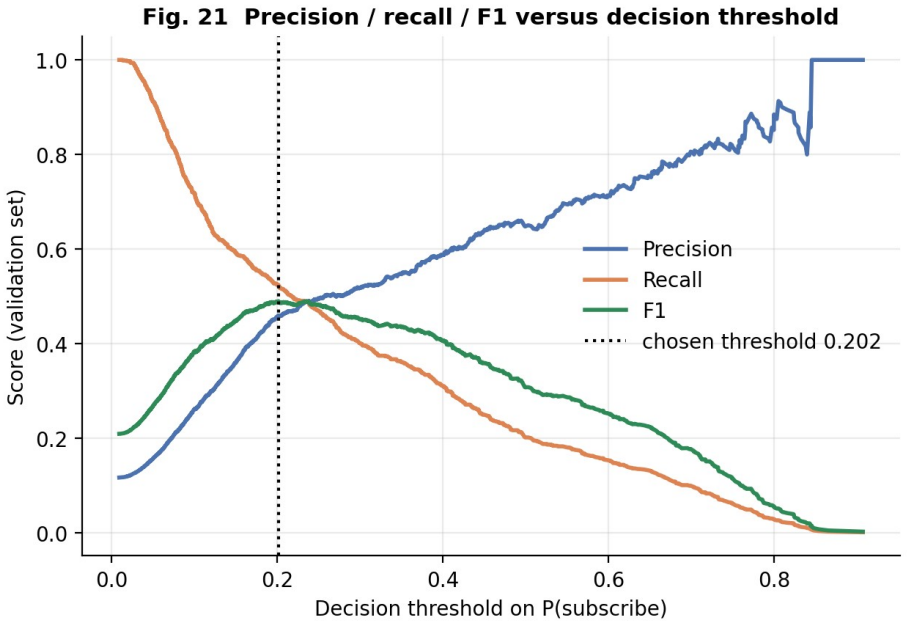


Figure 21. Threshold trade-off (validation set).

Question	How does the decision threshold trade precision against recall?
What it shows	Precision, recall and F1 as functions of the cut-off on predicted probability, with the chosen threshold.
Why this chart	The threshold is a business decision; showing the full trade-off makes the chosen operating point transparent.
Variables	Threshold, precision, recall, F1 (validation).
Finding	F1 is maximised at threshold 0.202; the same threshold is applied unchanged to the test set (test F1 0.457).
Interpretation	Raising the threshold buys precision at the cost of recall; a bank with expensive calls would pick a higher cut-off, and one with cheap calls a lower one.
Caveats	Chosen on validation only; F1 is not cost-sensitive.

Table 26. Permutation importance on the validation set

feature	mean AUC drop	SD (5 repeats)
contact	0.0767	0.0055
month	0.0685	0.0038
poutcome	0.0278	0.0018
day	0.0222	0.0024
pdays_clean	0.0153	0.0016
housing	0.0105	0.0018
marital	0.0090	0.0013
previous	0.0073	0.0007
campaign	0.0039	0.0012
balance	0.0037	0.0019
job	0.0032	0.0009
age	0.0028	0.0008
education	0.0017	0.0013
loan	0.0006	0.0006
previously_contacted	0.0004	0.0003
default	-0.0001	0.0003

6.4 Robustness: leakage and temporal shift

Fig. 22 Robustness: the MLP and baselines under different feature sets and split protocols

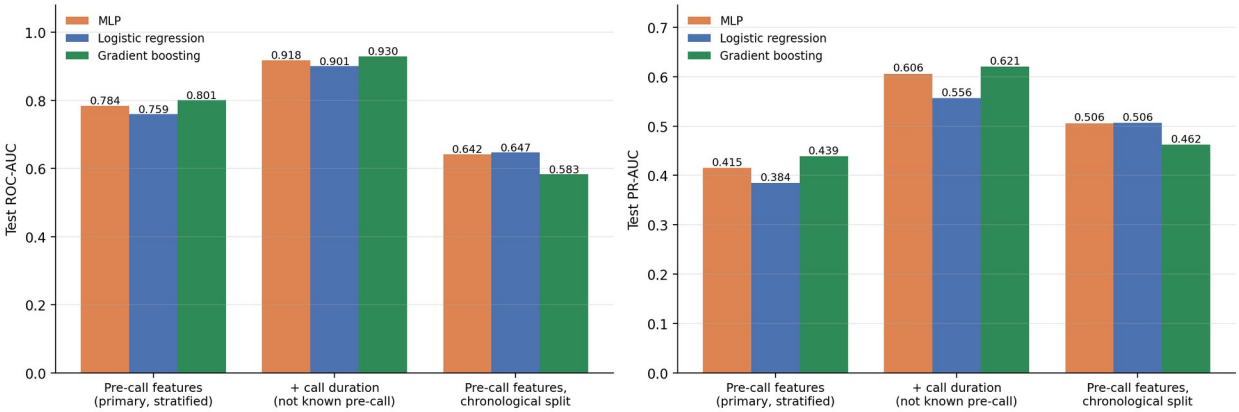


Figure 22. Robustness: leakage and temporal shift.

Question	How trustworthy is the headline result under realistic changes (post-call variable, time-ordered split)?
What it shows	Test ROC-AUC and PR-AUC of the MLP, logistic regression and gradient boosting under three protocols.
Why this chart	A single random split can be optimistic; contrasting protocols shows what the score really means.
Variables	Feature set (with/without duration), split type (stratified/chronological).
Finding	Adding call duration raises the MLP's test ROC-AUC from 0.784 to 0.918. On a chronological split the pre-call MLP scores ROC-AUC 0.642 (logistic regression 0.647, gradient boosting 0.583); subscription prevalence shifts from 5.8% (train) to 39.3% (test).
Interpretation	Duration is a strong but unusable signal (it is known only after the call), so the primary model correctly excludes it; the large jump shows how an unnoticed leak would inflate performance. Under the time-ordered split all three models fall to a ROC-AUC of about 0.58-0.65, and the ranking of models changes (gradient boosting, best on the random split, is worst), so the random-split score is an optimistic description of how the model would perform on a later campaign with a different client mix.
Caveats	The duration and chronological runs use a single seed (42) with the selected configuration; PR-AUC is not comparable across splits with different prevalence.

Table 27. Robustness experiments (primary row = mean of 5 seeds; others use seed 42)

Setting	Model	Test ROC-AUC	Test PR-AUC	Test F1	Test prevalence
Pre-call features (primary, stratified)	MLP	0.784	0.415	0.461	0.117
Pre-call features (primary, stratified)	Logistic regression	0.759	0.384	0.431	0.117
Pre-call features (primary, stratified)	Gradient boosting	0.801	0.439	0.476	0.117
+ call duration (not known pre-call)	MLP	0.918	0.606	0.594	0.117
+ call duration (not known pre-call)	Logistic regression	0.901	0.556	0.559	0.117
+ call duration (not known pre-call)	Gradient boosting	0.930	0.621	0.621	0.117
Pre-call features, chronological split	MLP	0.642	0.506	0.523	0.393
Pre-call features, chronological split	Logistic regression	0.647	0.506	0.401	0.393
Pre-call features, chronological split	Gradient boosting	0.583	0.462	0.554	0.393

6.5 Error analysis

- False positives (497) and false negatives (411) are of similar size; the model still misses 52% of subscribers (Figure 18).
- At the chosen threshold the precision of 43% is 3.7 times the base rate, so contacting the top-scored clients is more efficient than random calling but still wastes more than half of the calls.

- The precision-recall curve (Figure 19) shows precision falling quickly as recall rises, so a high-precision shortlist is feasible only for a small share of subscribers.
- Most of the model's discriminative power comes from timing (month, day) and contact-channel variables, i.e. information about how and when the campaign was run, which limits transfer to a different campaign (Figure 22).
- No overfitting was observed (training and validation losses coincide, Figure 16); the limiting factor is the information in the pre-call features.

7. Conclusion and Future Scope

7.1 What was learned

- The dataset is a clean-looking but information-hiding table: no NaN cells, yet hidden missingness in poutcome, contact and education and a -1 sentinel in pdays.
- Outcomes are imbalanced (11.7% positive) and driven mainly by the calling month, the previous-campaign outcome, the contact channel and loan status; individual numeric variables are only weakly related to the outcome.

7.2 Model performance

- A 32-unit one-hidden-layer MLP (1,697 parameters) reached test ROC-AUC 0.784 +/- 0.002, PR-AUC 0.415, F1 0.461 over five seeds (primary seed: ROC-AUC 0.784, F1 0.457, recall 0.48, precision 0.43).
- It outperformed logistic regression (0.759) but not gradient boosting (0.801); there was no overfitting, and the model is limited by the information available before a call.

7.3 Challenges

- Hidden missing values and a sentinel code; heavy-tailed variables; multicollinearity among the history variables (VIF 1005 before, 5.2 after).
- A documented leakage trap (duration) and a strong temporal shift (subscription rate 5.8% in the first 70% of the file vs 39.3% in the last 15%).
- Sandbox limits: UCI was unreachable, so a verified mirror was used (download code tries UCI first); no GPU, so the network was kept small.

7.4 Practical relevance and limitations

The analysis suggests concrete campaign levers: prioritise clients with a previous successful outcome, avoid mass-calling in the busiest months without targeting, limit repeated re-dialling (conversion falls from 14.6% at one contact to 3.9% beyond ten), and treat students, retirees and clients without housing loans as receptive segments. However, the findings are associational, the data come from one bank and an older campaign period, and the model's modest precision and the temporal shift mean it should support, not replace, human judgement. A high test score alone would not justify deployment.

7.5 Future scope

- Add the richer UCI 'bank-additional' version (20 inputs including macro-economic indicators such as the Euribor rate) and agent/call-direction features.
- Model count variables with Negative-Binomial/hurdle regressions and age non-linearly (splines); try gradient boosting with calibration and cost-sensitive thresholds.

- Evaluate with rolling-origin (time-series) validation, report uncertainty with bootstrapped intervals, and estimate uplift/profit curves instead of F1.
- Before any deployment: fairness review (age and job effects), monitoring for drift, and a pilot A/B test; a high AUC alone does not justify deployment.

8. References

(APA 7th edition. Authors, titles, venues, volumes, pages and DOIs below were checked for plausibility; Moro et al. (2014) and the UCI description were verified online while preparing this report. Please verify the remaining entries on the publisher pages before submission.)

Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. arXiv:1603.04467.

Fry, B. (2007). Visualizing data: Exploring and explaining data with the Processing environment. O'Reilly Media. [Source of the seven stages: acquire, parse, filter, mine, represent, refine, interact.]

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track). arXiv:2207.08815.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations. arXiv:1412.6980.

Moro, S., Cortez, P., & Rita, P. (2014). A data-driven approach to predict the success of bank telemarketing. Decision Support Systems, 62, 22-31. https://doi.org/10.1016/j.dss.2014.03.001

Moro, S., Rita, P., & Cortez, P. (2012). Bank Marketing [Data set]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/222/bank+marketing

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1), 1929-1958.

Tukey, J. W. (1977). Exploratory data analysis. Addison-Wesley.

Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 261-272. https://doi.org/10.1038/s41592-019-0686-2

Waskom, M. L. (2021). seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021. https://doi.org/10.21105/joss.03021

Relevance: Moro et al. (2014) and the UCI page document the data and prior modelling; Fry (2007) and Tukey (1977) provide the methodological frame; Pedregosa, Virtanen, Hunter, Waskom and Abadi document the software used; Kingma & Ba and Srivastava et al. justify the optimiser and dropout regularisation; Saito & Rehmsmeier justify PR-based evaluation; Grinsztajn et al. motivate the gradient-boosting reference.

9. Appendix

A. Rubric compliance matrix

Table A1. Rubric matrix (40 marks). No grade is guaranteed; the matrix shows how each criterion is addressed

Requirement	Max	Implementation evidence	File / section	Validation status	Remaining work
CO3 Selection and creation of visualizations	3	22 figures of 12+ distinct types chosen per data type: bar, histogram, boxplot, scatter/pairplot, violin, heatmaps, Q-Q, PMF overlay, forest, ROC/PR, confusion matrix, learning curves.	Report Sec. 4 & 6; src/visualizations.py, src/evaluate.py; outputs/figures/	Generated by code and inspected visually	None
CO3 Quality and presentation	2	Titles stating the question, labelled axes with units, log/symlog scales where needed, consistent colours, CIs, n annotations, no 3-D/dual axes, 200 dpi.	outputs/figures/*.png; visualization stage 6 (Refine)	Visual QA performed; several defects found and fixed (binning, clipped labels, scales)	None
CO3 Interpretation of visualizations	3	Every figure has: question, what it shows, why chosen, variables, finding (with computed statistic), interpretation, caveats.	Report Sec. 4 & 6 figure tables; src/narrative.py	Numbers generated from results; model claims checked against actual output	None
CO4 Univariate analysis and distribution exploration	4	Count, mean, median, mode, min/max, range, variance, SD, quartiles, IQR, skewness, kurtosis; frequency tables; histograms, boxplots, bar charts.	Report Sec. 2.3, 3.2, 4.1; outputs/tables/numeric_summary.csv, freq_*.csv; Figs. 1-4	Validated by tests (IQR, summary) and re-computation	None
CO4 Discrete / continuous probability distributions	4	Bernoulli (target, with exact CI and month homogeneity test); Poisson vs Negative Binomial (campaign, previous); Normal/Lognormal/Gamma (age); Exponential/Lognormal/Gamma/Weibull (duration); MLE, AIC, KS distance, chi-square, Q-Q plots.	Report Sec. 4.2; src/eda.py; Figs. 7-9	Fitting code tested on synthetic data with known truth	None
CO4 Bivariate / multivariate analysis	4	Spearman vs Pearson, Mann-Whitney + Cliff's delta, Kruskal-Wallis, chi-square + Cramer's V, pairplot, grouped violin, multivariable logistic regression with VIF diagnostics.	Report Sec. 4.3; Figs. 10-14; outputs/tables/	Multicollinearity found and corrected (VIF 1005 -> 5.2)	None
CO4 Interpretation of patterns and statistical findings	4	Ten key findings each with observation, statistic, chart, implication, caveat; association is distinguished from causation.	Report Sec. 4.4 (Table 12)	Numbers computed, not typed	None
CO5 Real-world problem and dataset relevance	3	Bank telemarketing for term deposits; real Portuguese bank data (UCI #222), 16 genuine features, 45,211 records; dataset comparison table.	Report Sec. 1, 2	Dataset verified (shape, columns, SHA-256)	None
CO5 Appropriate EDA, transformation and analysis	3	Hidden missingness, sentinel, duplicates, outliers with retain/transform decisions, leakage-safe preprocessing fitted on train only, stratified + chronological splits, before/after comparison.	Report Sec. 3; src/preprocessing.py	30 automated tests incl. leakage tests	None
CO5 Meaningful findings and interpretation	3	Findings on timing, previous outcome, age U-shape, loans, over-dispersion; model leakage and temporal-shift experiments.	Report Sec. 4.4, 6	Checked against executed results	None
CO5 Conclusions, recommendations, limitations, future scope	3	Conclusions, limitations (association not causation, temporal shift, no year field), realistic future work, no deployment claim.	Report Sec. 7	Written from actual results	None
CO5 GitHub repository and documentation	4	Structured repository, README, requirements, notebook + src modules, tests, dataset (with download script) and data card, report PDF/DOCX, rubric checklist.	Repository root; README.md; reports/	Prepared and tested locally	MANUAL: push to your own GitHub account, make it public, submit the link in the Google Form

B. Reproducibility and execution

- Install: pip install -r requirements.txt (Python 3.10+; tested with Python 3.12, TensorFlow-CPU 2.21).
- Full pipeline: python scripts/run_all.py (about 10-15 minutes on one CPU core); quick smoke test: python scripts/run_all.py --quick (code check only, NOT a full run).
- Tests: pytest -q (30 tests, all passing at the time of writing).
- Notebook: notebooks/eda_and_modeling.ipynb (executed; model section loads the saved artefacts, with a switch to retrain).
- Interactive explorer: outputs/interactive/explorer.html.

C. Source code

The complete source code is reproduced below (also in the repository under src/ and scripts/).

src/config.py

```
"""Central configuration: paths, seeds, column groups, dataset metadata."""
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
RAW_CSV = DATA_DIR / "raw" / "bank-full.csv"
OUT = ROOT / "outputs"
FIG_DIR = OUT / "figures"
TAB_DIR = OUT / "tables"
RES_DIR = OUT / "results"
MODEL_DIR = OUT / "models"
for _d in (FIG_DIR, TAB_DIR, RES_DIR, MODEL_DIR, OUT / "interactive"):
    _d.mkdir(parents=True, exist_ok=True)

SEED = 42

# --- Dataset metadata (UCI Machine Learning Repository, dataset id 222) ---
UCI_PAGE = "https://archive.ics.uci.edu/dataset/222/bank+marketing"
UCI_ZIP = "https://archive.ics.uci.edu/static/public/222/bank+marketing.zip"
# Fallback mirror of the same file (used when UCI is unreachable, e.g. on restricted networks)
MIRROR_CSV = "https://raw.githubusercontent.com/samikabir/BankMarketingPrediction/master/bank-full.csv"
EXPECTED_SHAPE = (45211, 17)  # as documented by UCI: 45,211 instances = 16 inputs + 1 target

TARGET = "y"
NUMERIC = ["age", "balance", "day", "duration", "campaign", "pdays", "previous"]
CATEGORICAL = ["job", "marital", "education", "default", "housing", "loan",
               "contact", "month", "poutcome"]
MONTH_ORDER = ["jan", "feb", "mar", "apr", "may", "jun",
               "jul", "aug", "sep", "oct", "nov", "dec"]

# Model feature groups. `duration` is excluded from the primary ("pre-call") model: it is only
# known after the call ends, so using it would leak the outcome in any realistic deployment.
MODEL_NUMERIC = ["age", "balance", "day", "campaign", "pdays_clean", "previous"]
MODEL_BINARY = ["previously_contacted"]
```

src/data_loader.py

```
"""Stage 1 (Acquire): download, verify and load the Bank Marketing dataset."""
import hashlib
import io
import urllib.request
import zipfile
from pathlib import Path

import pandas as pd

from .config import (CATEGORICAL, EXPECTED_SHAPE, MIRROR_CSV, NUMERIC, RAW_CSV, TARGET,
                     UCI_ZIP)


def _fetch(url: str, timeout: int = 60) -> bytes:
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0 (EDA assignment)"})
    with urllib.request.urlopen(req, timeout=timeout) as r:
        return r.read()


def _csv_from_uci_zip(blob: bytes) -> bytes:
    """UCI wraps the data in nested zips; search recursively for bank-full.csv."""
    with zipfile.ZipFile(io.BytesIO(blob)) as z:
        for name in z.namelist():
            if name.endswith("bank-full.csv"):
                return z.read(name)
        for name in z.namelist():
            if name.endswith(".zip"):
                try:
                    return _csv_from_uci_zip(z.read(name))
                except FileNotFoundError:
                    continue
    raise FileNotFoundError("bank-full.csv not found inside archive")


def sha256(path: Path) -> str:
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()


def download_dataset(force: bool = False, dest: Path = RAW_CSV, verbose: bool = True) -> Path:
    """Download bank-full.csv to `dest`. Tries the official UCI archive first, then a mirror.

    Returns the local path. Raises RuntimeError with manual instructions if both sources fail.
    """
    dest = Path(dest)
    if dest.exists() and not force:
        return dest
    dest.parent.mkdir(parents=True, exist_ok=True)
    errors = []
```

```
    sources = (("UCI archive", lambda: _csv_from_uci_zip(_fetch(UCI_ZIP))),
               ("GitHub mirror", lambda: _fetch(MIRROR_CSV)))
    for label, fn in sources:
        try:
            dest.write_bytes(fn())
            validate(load_raw(dest))  # fail early if the file is not what we expect
            if verbose:
                print(f"Downloaded dataset from {label}: {dest}")
            return dest
        except Exception as e:  # noqa: BLE001 - try the fallback on any failure
            errors.append(f"{label}: {e!r}")
            dest.unlink(missing_ok=True)
    raise RuntimeError(
        "Could not download the dataset automatically.\n" + "\n".join(errors) +
        "\nManual fallback: download 'bank-full.csv' from "
        "https://archive.ics.uci.edu/dataset/222/bank+marketing and place it at " + str(dest))


def load_raw(path: Path = RAW_CSV) -> pd.DataFrame:
    """Load the semicolon-separated CSV exactly as published (no cleaning)."""
    return pd.read_csv(path, sep=";")


def validate(df: pd.DataFrame) -> None:
    """Sanity checks that the file is the documented Bank Marketing dataset."""
    if df.shape != EXPECTED_SHAPE:
        raise ValueError(f"Unexpected shape {df.shape}, expected {EXPECTED_SHAPE}")
    expected = set(NUMERIC + CATEGORICAL + [TARGET])
    if set(df.columns) != expected:
        raise ValueError(f"Unexpected columns: {set(df.columns) ^ expected}")
    if set(df[TARGET].unique()) != {"yes", "no"}:
        raise ValueError("Target column must contain only 'yes'/'no'")


def get_data() -> pd.DataFrame:
    """Ensure the file exists, load and validate it."""
    df = load_raw(download_dataset())
    validate(df)
    return df


if __name__ == "__main__":
    p = download_dataset()
    d = load_raw(p)
    validate(d)
    print(d.shape, "sha256:", sha256(p))
```

src/preprocessing.py

```
"""Stage 3 (Filter): cleaning, feature construction, leakage-safe preprocessing and splitting."""
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler

from .config import CATEGORICAL, MODEL_BINARY, SEED, TARGET


# ---------------------------------------------------------------------------
# Deterministic cleaning (no statistics are learned here, so it cannot leak across splits)
# ---------------------------------------------------------------------------
def clean(df: pd.DataFrame) -> pd.DataFrame:
    """Standardise text, encode the target and make the pdays sentinel explicit.

    * 'unknown' categories are KEPT as their own level: they are informative (e.g. contact =
      'unknown' is strongly associated with the outcome) and are not random missingness.
    * pdays == -1 is a sentinel meaning "never contacted before", not a real duration, so it is
      split into a binary flag + a cleaned numeric column (0 where not contacted).
    """
    out = df.copy()
    for c in CATEGORICAL + [TARGET]:
        out[c] = out[c].astype(str).str.strip().str.lower()
    out["y_bin"] = (out[TARGET] == "yes").astype(int)
    out["previously_contacted"] = (out["pdays"] != -1).astype(int)
    out["pdays_clean"] = out["pdays"].where(out["pdays"] != -1, 0)
    return out


# ---------------------------------------------------------------------------
# Learned transformers (fit on TRAIN ONLY)
# ---------------------------------------------------------------------------
def signed_log1p(x):
    """Symmetric log transform: keeps negative balances (valid overdrafts) usable."""
    x = np.asarray(x, dtype=float)
    return np.sign(x) * np.log1p(np.abs(x))


class Winsorizer(BaseEstimator, TransformerMixin):
    """Cap values at an upper quantile learned from the training data."""

    def __init__(self, upper_q: float = 0.99):
```

```
        self.upper_q = upper_q

    def fit(self, X, y=None):
        self.upper_ = np.quantile(np.asarray(X, dtype=float), self.upper_q, axis=0)
        self.n_features_in_ = np.asarray(X).shape[1]
        return self

    def transform(self, X):
        return np.minimum(np.asarray(X, dtype=float), self.upper_)

    def get_feature_names_out(self, input_features=None):
        return np.asarray(input_features)


def build_preprocessor(extra_log_capped=None) -> ColumnTransformer:
    """ColumnTransformer used by the neural network and the baselines.

    age, day            -> StandardScaler
    balance             -> signed log1p -> StandardScaler   (heavy tails, negatives allowed)
    campaign, previous  -> cap at train 99th pct -> log1p -> StandardScaler   (count outliers)
    pdays_clean         -> log1p -> StandardScaler
    categorical         -> OneHotEncoder(handle_unknown='ignore')
    binary flag         -> passthrough
    """
    log_capped = ["campaign", "previous"] + list(extra_log_capped or [])
    capped = Pipeline([("cap", Winsorizer(0.99)),
                       ("log", FunctionTransformer(np.log1p, feature_names_out="one-to-one")),
                       ("sc", StandardScaler())])
    slog = Pipeline([("log", FunctionTransformer(signed_log1p, feature_names_out="one-to-one")),
                     ("sc", StandardScaler())])
    plog = Pipeline([("log", FunctionTransformer(np.log1p, feature_names_out="one-to-one")),
                     ("sc", StandardScaler())])
    return ColumnTransformer([
        ("plain", StandardScaler(), ["age", "day"]),
        ("slog", slog, ["balance"]),
        ("capped", capped, log_capped),
        ("pdays", plog, ["pdays_clean"]),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), CATEGORICAL),
        ("bin", "passthrough", MODEL_BINARY),
    ], verbose_feature_names_out=False)


def model_columns(include_duration: bool = False):
    cols = ["age", "balance", "day", "campaign", "pdays_clean", "previous"]
    cols += CATEGORICAL + MODEL_BINARY
    if include_duration:
        cols = cols + ["duration"]
    return cols


# ---------------------------------------------------------------------------------------
# Splitting
# ---------------------------------------------------------------------------------------
def split_data(df: pd.DataFrame, kind: str = "stratified", seed: int = SEED,
               val_frac: float = 0.15, test_frac: float = 0.15):
    """Return (train, val, test) DataFrames.

    kind='stratified'    : random 70/15/15 split stratified on the target (primary protocol).
    kind='chronological' : the UCI file is ordered by date (May 2008 - Nov 2010); first 70 % ->
                           train, next 15 % -> validation, last 15 % -> test (robustness check).
    """
    if kind == "stratified":
        trainval, test = train_test_split(df, test_size=test_frac, stratify=df["y_bin"],
                                          random_state=seed)
        rel_val = val_frac / (1 - test_frac)
        train, val = train_test_split(trainval, test_size=rel_val, stratify=trainval["y_bin"],
                                      random_state=seed)
    elif kind == "chronological":
        n = len(df)
        i_val, i_test = int(n * (1 - val_frac - test_frac)), int(n * (1 - test_frac))
        train, val, test = df.iloc[:i_val], df.iloc[i_val:i_test], df.iloc[i_test:]
    else:
        raise ValueError(kind)
    return train.copy(), val.copy(), test.copy()


def prepare_matrices(train, val, test, include_duration: bool = False):
    """Fit the preprocessor on TRAIN only, then transform train / val / test."""
    cols = model_columns(include_duration)
    pre = build_preprocessor(extra_log_capped=["duration"] if include_duration else None)
    Xtr = pre.fit_transform(train[cols])
    Xva, Xte = pre.tra
nsform(val[cols]), pre.transform(test[cols])
    names = list(pre.get_feature_names_out())
    return (Xtr.astype("float32"), Xva.astype("float32"), Xte.astype("float32"),
            train["y_bin"].values, val["y_bin"].values, test["y_bin"].values, pre, names)
```

src/eda.py

```
"""Stage 2 (Parse) & Stage 4 (Mine): data quality, univariate, distribution, bivariate and
multivariate statistics. Every number reported in the report is produced here from the data."""
import json

import numpy as np
```

```
import pandas as pd
import statsmodels.api as sm
from scipy import stats
from scipy.optimize import minimize_scalar

from .config import CATEGORICAL, MONTH_ORDER, NUMERIC, RES_DIR, TAB_DIR, TARGET
from .preprocessing import signed_log1p


# ---------------------------------------------------------------------------------------
# Structure & data-quality
# ---------------------------------------------------------------------------------------
def structure_report(raw: pd.DataFrame) -> pd.DataFrame:
    rep = pd.DataFrame({"dtype": raw.dtypes.astype(str), "n_unique": raw.nunique(),
                        "n_missing_NaN": raw.isna().sum(),
                        "pct_missing_NaN": (raw.isna().mean() * 100).round(3)})
    rep["memory_KB"] = (raw.memory_usage(deep=True, index=False) / 1024).round(1)
    return rep


def quality_report(raw: pd.DataFrame) -> pd.DataFrame:
    """NaN is not the only way a value can be missing: look for 'unknown' labels and sentinels."""
    rows = []
    for c in CATEGORICAL:
        n_unk = int((raw[c].astype(str).str.lower() == "unknown").sum())
        rows.append({"column": c, "issue": "'unknown' label", "count": n_unk,
                     "pct": round(100 * n_unk / len(raw), 2)})
    n_sent = int((raw["pdays"] == -1).sum())
    rows.append({"column": "pdays", "issue": "sentinel -1 (never contacted)", "count": n_sent,
                 "pct": round(100 * n_sent / len(raw), 2)})
    n_neg = int((raw["balance"] < 0).sum())
    rows.append({"column": "balance", "issue": "negative balance (valid overdraft)", "count": n_neg,
                 "pct": round(100 * n_neg / len(raw), 2)})
    n_zero = int((raw["duration"] == 0).sum())
    rows.append({"column": "duration", "issue": "zero-second calls", "count": n_zero,
                 "pct": round(100 * n_zero / len(raw), 3)})
    rows.append({"column": "(all)", "issue": "exact duplicate rows", "count": int(raw.duplicated().sum()),
                 "pct": round(100 * raw.duplicated().mean(), 3)})
    rows.append({"column": "(all)", "issue": "NaN cells", "count": int(raw.isna().sum().sum()), "pct": 0.0})
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------------------
# Univariate
# ---------------------------------------------------------------------------------------
def numeric_summary(df: pd.DataFrame, cols) -> pd.DataFrame:
    out = {}
    for c in cols:
        s = df[c].dropna()
        q1, q3 = s.quantile([.25, .75])
        out[c] = {"count": int(s.count()), "missing": int(df[c].isna().sum()), "mean": s.mean(),
                  "median": s.median(), "mode": s.mode().iloc[0], "min": s.min(), "max": s.max(),
                  "range": s.max() - s.min(), "variance": s.var(), "std": s.std(), "Q1": q1, "Q3": q3,
                  "IQR": q3 - q1, "skewness": stats.skew(s), "excess_kurtosis": stats.kurtosis(s)}
    return pd.DataFrame(out).T


def iqr_outliers(df: pd.DataFrame, cols) -> pd.DataFrame:
    rows = []
    for c in cols:
        s = df[c]
        q1, q3 = s.quantile([.25, .75])
        iqr = q3 - q1
        lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
        n_lo, n_hi = int((s < lo).sum()), int((s > hi).sum())
        far_hi = q3 + 3 * iqr
        rows.append({"variable": c, "lower_fence": lo, "upper_fence": hi, "n_below": n_lo,
                     "n_above": n_hi, "pct_outliers": 100 * (n_lo + n_hi) / len(s),
                     "n_extreme(>3IQR)": int((s > far_hi).sum() + (s < q1 - 3 * iqr).sum()),
                     "max": s.max(), "p99": s.quantile(.99)})
    return pd.DataFrame(rows)


def categorical_frequency(df: pd.DataFrame, col: str) -> pd.DataFrame:
    vc = df[col].value_counts()
    return pd.DataFrame({"count": vc, "percent": (100 * vc / vc.sum()).round(2)})


def rate_table(df: pd.DataFrame, col: str, order=None) -> pd.DataFrame:
    """Subscription rate per category with exact (Clopper-Pearson) 95 % confidence intervals."""
    g = df.groupby(col)["y_bin"].agg(["sum", "count"])
    g["rate"] = g["sum"] / g["count"]
    ci = [stats.binomtest(int(k), int(n)).proportion_ci(0.95, method="exact") for k, n in zip(g["sum"], g["count"])]
    g["ci_low"], g["ci_high"] = [c.low for c in ci], [c.high for c in ci]
    g = g.rename(columns={"sum": "subscribed", "count": "n"})
    if order is not None:
        g = g.reindex(order)
    return g


# ---------------------------------------------------------------------------------------
# Probability distributions
# ---------------------------------------------------------------------------------------
def bernoulli_summary(df: pd.DataFrame) -> dict:
    k, n = int(df["y_bin"].sum()), len(df)
    ci = stats.binomtest(k, n).proportion_ci(0.95, method="exact")
    # Is one common Bernoulli(p) plausible across months? (homogeneity chi-square)
```

```
    ct = pd.crosstab(df["month"], df["y_bin"])
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    monthly = df.groupby("month")["y_bin"].mean()
    return {"k": k, "n": n, "p_hat": k / n, "ci_low": ci.low, "ci_high": ci.high,
            "variance_p_1_minus_p": (k / n) * (1 - k / n),
            "month_homogeneity_chi2": chi2, "month_homogeneity_dof": int(dof), "month_homogeneity_p": p,
            "monthly_rate_min": float(monthly.min()), "monthly_rate_max": float(monthly.max())}


def _nb_fit(x: np.ndarray):
    """MLE of the negative binomial (mean mu = sample mean, size r)."""
    mu = x.mean()

    def nll(log_r):
        r = np.exp(log_r)
        return -stats.nbinom.logpmf(x, r, r / (r + mu)).sum()

    res = minimize_scalar(nll, bounds=(np.log(1e-2), np.log(1e4)), method="bounded")
    r = float(np.exp(res.x))
    return r, r / (r + mu), -res.fun


def count_distribution_fit(x: np.ndarray, name: str, max_bin: int = 9) -> dict:
    """C
ompare Poisson and Negative-Binomial models for a non-negative count variable."""
    x = np.asarray(x, dtype=int)
    n, mu, var = len(x), x.mean(), x.var(ddof=1)
    r, p, ll_nb = _nb_fit(x)
    ll_pois = stats.poisson.logpmf(x, mu).sum()
    ks = np.arange(0, max_bin + 1)
    obs = np.array([(x == k).sum() for k in ks[:-1]] + [(x >= max_bin).sum()])
    exp_p = np.append(stats.poisson.pmf(ks[:-1], mu), stats.poisson.sf(max_bin - 1, mu)) * n
    exp_n = np.append(stats.nbinom.pmf(ks[:-1], r, p), stats.nbinom.sf(max_bin - 1, r, p)) * n
    chi_p, chi_n = ((obs - exp_p) ** 2 / exp_p).sum(), ((obs - exp_n) ** 2 / exp_n).sum()
    dof_p, dof_n = len(obs) - 1 - 1, len(obs) - 1 - 2
    return {"variable": name, "n": n, "mean": mu, "variance": var, "dispersion_index_var_over_mean": var / mu,
            "poisson_lambda": mu, "poisson_loglik": ll_pois, "poisson_AIC": 2 * 1 - 2 * ll_pois,
            "poisson_chi2": chi_p, "poisson_dof": dof_p, "poisson_p": stats.chi2.sf(chi_p, dof_p),
            "nb_r": r, "nb_p": p, "nb_loglik": ll_nb, "nb_AIC": 2 * 2 - 2 * ll_nb,
            "nb_chi2": chi_n, "nb_dof": dof_n, "nb_p_value": stats.chi2.sf(chi_n, dof_n),
            "obs_zero_pct": 100 * (x == 0).mean(), "poisson_zero_pct": 100 * stats.poisson.pmf(0, mu),
            "nb_zero_pct": 100 * stats.nbinom.pmf(0, r, p), "max_obs": int(x.max()),
            "prob_ge_10_obs_pct": 100 * (x >= 10).mean(), "prob_ge_10_poisson_pct": 100 * stats.poisson.sf(9, mu),
            "prob_ge_10_nb_pct": 100 * stats.nbinom.sf(9, r, p)}


CONT_DISTS = {"normal": stats.norm, "lognormal": stats.lognorm, "gamma": stats.gamma,
              "exponential": stats.expon, "weibull": stats.weibull_min}


def continuous_fits(x: np.ndarray, names, positive_support=True) -> pd.DataFrame:
    """Fit candidate continuous distributions by MLE; compare with AIC and the KS distance.

    With n ~ 45,000 *any* imperfect model is rejected by a KS test (tiny p-values), so the KS
    statistic D (the maximum CDF gap) is read as an effect size, not as a pass/fail verdict.
    """
    rows = []
    x = np.asarray(x, dtype=float)
    for nm in names:
        d = CONT_DISTS[nm]
        params = d.fit(x) if nm == "normal" else d.fit(x, floc=0)
        ll = d.logpdf(x, *params).sum()
        k = 2 if nm in ("normal", "gamma", "lognormal", "weibull") else 1  # free parameters
        ks = stats.kstest(x, d.cdf, args=params)
        rows.append({"distribution": nm, "params": tuple(np.round(params, 4)), "loglik": ll,
                     "AIC": 2 * k - 2 * ll, "KS_D": ks.statistic, "KS_p": ks.pvalue})
    out = pd.DataFrame(rows).sort_values("AIC").reset_index(drop=True)
    out["delta_AIC"] = out["AIC"] - out["AIC"].min()
    return out


def normality_tests(x: np.ndarray, seed=42) -> dict:
    x = np.asarray(x, dtype=float)
    k2, p_k2 = stats.normaltest(x)
    sample = np.random.default_rng(seed).choice(x, size=5000, replace=False)
    sw, p_sw = stats.shapiro(sample)
    return {"dagostino_K2": k2, "dagostino_p": p_k2, "shapiro_W_n5000": sw, "shapiro_p": p_sw,
            "skew": float(stats.skew(x)), "excess_kurtosis": float(stats.kurtosis(x))}


# ---------------------------------------------------------------------------------------
# Bivariate / multivariate
# ---------------------------------------------------------------------------------------
def cramers_v(a: pd.Series, b: pd.Series) -> float:
    """Bias-corrected Cramer's V (Bergsma, 2013)."""
    ct = pd.crosstab(a, b)
    chi2 = stats.chi2_contingency(ct, correction=False)[0]
    n = ct.values.sum()
    phi2 = chi2 / n
    r, k = ct.shape
    phi2c = max(0, phi2 - (k - 1) * (r - 1) / (n - 1))
    rc, kc = r - (r - 1) ** 2 / (n - 1), k - (k - 1) ** 2 / (n - 1)
    denom = min(kc - 1, rc - 1)
    return float(np.sqrt(phi2c / denom)) if denom > 0 else 0.0


def cramers_matrix(df: pd.DataFrame, cols) -> pd.DataFrame:
```

```
    m = pd.DataFrame(index=cols, columns=cols, dtype=float)
    for a in cols:
        for b in cols:
            m.loc[a, b] = 1.0 if a == b else cramers_v(df[a], df[b])
    return m


def chi2_vs_target(df: pd.DataFrame, cols) -> pd.DataFrame:
    rows = []
    for c in cols:
        ct = pd.crosstab(df[c], df[TARGET])
        chi2, p, dof, _ = stats.chi2_contingency(ct)
        rows.append({"variable": c, "chi2": chi2, "dof": int(dof), "p_value": p,
                     "cramers_v": cramers_v(df[c], df[TARGET])})
    return pd.DataFrame(rows).sort_values("cramers_v", ascending=False).reset_index(drop=True)


def mannwhitney_vs_target(df: pd.DataFrame, cols) -> pd.DataFrame:
    """Rank-based group comparison (robust to the heavy tails seen in these variables)."""
    rows = []
    yes, no = df[df[TARGET] == "yes"], df[df[TARGET] == "no"]
    for c in cols:
        u, p = stats.mannwhitneyu(yes[c], no[c], alternative="two-sided")
        delta = 2 * u / (len(yes) * len(no)) - 1  # Cliff's delta (= 2*AUC - 1)
        rows.append({"variable": c, "median_yes": yes[c].median(), "median_no": no[c].median(),
                     "mean_yes": yes[c].mean(), "mean_no": no[c].mean(), "U": u, "p_value": p,
                     "cliffs_delta": delta})
    return pd.DataFrame(rows).sort_values("cliffs_delta", key=np.abs, ascending=False).reset_index(drop=True)


def kruskal_effect(df: pd.DataFrame, value: str, group: str) -> dict:
    groups = [g[value].values for _, g in df.groupby(group)]
    h, p = stats.kruskal(*groups)
    k, n = len(groups), len(df)
    return {"value": value, "group": group, "H": h, "p_value": p, "epsilon_sq": (h - k + 1) / (n - k)}


def correlation_matrices(df: pd.DataFrame, cols) -> dict:
    return {"spearman": df[cols].corr(method="spearman"), "pearson": df[cols].corr(method="pearson")}


def _logit_design(train: pd.DataFrame, drop_redundant: bool) -> pd.DataFrame:
    """Design matrix for the descriptive logistic regression (pre-call information only).

    `previous`, `pdays` and `previously_contacted` all describe the same thing (earlier
    contac
ts), so with drop_redundant=True only log1p(previous) is kept for that concept.
    """
    X = pd.DataFrame({
        "age_per_10y": train["age"] / 10, "balance_signedlog": signed_log1p(train["balance"]),
        "day": train["day"], "log1p_campaign": np.log1p(train["campaign"]),
        "log1p_previous": np.log1p(train["previous"])})
    if not drop_redundant:
        X["log1p_pdays"] = np.log1p(train["pdays_clean"])
        X["previously_contacted"] = train["previously_contacted"]
    for c in CATEGORICAL:
        ref = train[c].value_counts().index[0]  # reference level = most frequent category
        d = pd.get_dummies(train[c], prefix=c).astype(float)
        X = pd.concat([X, d.drop(columns=f"{c}_{ref}")], axis=1)
    return sm.add_constant(X.astype(float))


def vif_table(train: pd.DataFrame, drop_redundant: bool) -> pd.Series:
    """Variance inflation factors (VIF > 10 is the usual warning level)."""
    from statsmodels.stats.outliers_influence import variance_inflation_factor
    X = _logit_design(train, drop_redundant)
    v = {c: variance_inflation_factor(X.values, i) for i, c in enumerate(X.columns) if c != "const"}
    return pd.Series(v).sort_values(ascending=False)


def logit_odds_ratios(train: pd.DataFrame) -> pd.DataFrame:
    """Multivariable logistic regression (descriptive) on pre-call information.

    Odds ratios are *associations adjusted for the other variables*, not causal effects.
    """
    X = _logit_design(train, drop_redundant=True)
    fit = sm.Logit(train["y_bin"].astype(float), X).fit(disp=0, maxiter=200)
    ci = fit.conf_int()
    out = pd.DataFrame({"odds_ratio": np.exp(fit.params), "ci_low": np.exp(ci[0]), "ci_high": np.exp(ci[1]),
                        "p_value": fit.pvalues}).drop(index="const")
    out.attrs["pseudo_r2"] = float(fit.prsquared)
    out.attrs["n"] = int(fit.nobs)
    return out


def before_after_skew(train: pd.DataFrame, Xtr: np.ndarray, names) -> pd.DataFrame:
    """Skewness / excess kurtosis of heavy-tailed variables before and after the learned transforms."""
    rows = []
    for c in ["balance", "campaign", "previous", "pdays_clean"]:
        raw, new = train[c].values.astype(float), Xtr[:, list(names).index(c)]
        rows.append({"variable": c, "skew_before": stats.skew(raw), "skew_after": stats.skew(new),
                     "kurtosis_before": stats.kurtosis(raw), "kurtosis_after": stats.kurtosis(new)})
    return pd.DataFrame(rows)


def age_group(s: pd.Series) -> pd.Categorical:
    bins = [0, 24, 34, 44, 54, 64, 120]
```

```
    labels = ["<25", "25-34", "35-44", "45-54", "55-64", "65+"]
    return pd.cut(s, bins=bins, labels=labels)


# ---------------------------------------------------------------------------------------
# Orchestrator
# ---------------------------------------------------------------------------------------
def _clean_json(o):
    if isinstance(o, dict):
        return {str(k): _clean_json(v) for k, v in o.items()}
    if isinstance(o, (list, tuple)):
        return [_clean_json(v) for v in o]
    if isinstance(o, (np.integer,)):
        return int(o)
    if isinstance(o, (np.floating, float)):
        return None if (isinstance(o, float) and np.isnan(o)) else float(o)
    return o


def run_eda(raw: pd.DataFrame, cdf: pd.DataFrame, train: pd.DataFrame, save: bool = True) -> dict:
    """Run every statistical analysis and return a dictionary of results (also saved to disk)."""
    R = {}
    num_cols = ["age", "balance", "day", "duration", "campaign", "pdays", "previous"]

    tables = {
        "structure": structure_report(raw), "quality": quality_report(raw),
        "numeric_summary": numeric_summary(cdf, num_cols),
        "outliers_iqr": iqr_outliers(cdf, ["age", "balance", "duration", "campaign", "previous"]),
    }
    R["n_rows"], R["n_cols"] = int(raw.shape[0]), int(raw.shape[1])
    R["class_counts"] = cdf[TARGET].value_counts().to_dict()
    R["bernoulli"] = bernoulli_summary(cdf)

    # categorical frequency tables + subscription rates
    for c in CATEGORICAL:
        tables[f"freq_{c}"] = categorical_frequency(cdf, c)
        tables[f"rate_{c}"] = rate_table(cdf, c, order=MONTH_ORDER if c == "month" else None)
    cdf = cdf.assign(age_group=age_group(cdf["age"]))
    tables["rate_age_group"] = rate_table(cdf, "age_group")

    # discrete distributions
    R["campaign_counts"] = count_distribution_fit(cdf["campaign"].values - 1, "campaign-1 (extra contacts)")
    R["previous_counts"] = count_distribution_fit(cdf["previous"].values, "previous")

    # continuous distributions
    R["age_normality"] = normality_tests(cdf["age"].values)
    tables["fit_age"] = continuous_fits(cdf["age"].values, ["normal", "lognormal", "gamma"])
    dur = cdf.loc[cdf["duration"] > 0, "duration"].values
    R["duration_n_positive"] = int(len(dur))
    tables["fit_duration"] = continuous_fits(dur, ["exponential", "gamma", "weibull", "lognormal"])
    R["balance_normality"] = normality_tests(cdf["balance"].values)

    # bivariate
    corr_cols = ["age", "balance", "day", "duration", "campaign", "pdays_clean", "previous", "y_bin"]
    cm = correlation_matrices(cdf, corr_cols)
    tables["corr_spearman"], tables["corr_pearson"] = cm["spearman"], cm["pearson"]
    tables["mannwhitney"] = mannwhitney_vs_target(cdf, ["age", "balance", "day", "duration", "campaign", "pdays_clean",
"previous"])
    tables["chi2_target"] = chi2_vs_target(cdf, CATEGORICAL)
    tables["cramers_matrix"] = cramers_matrix(cdf, CATEGORICAL + [TARGET])
    R["kruskal_balance_job"] = kruskal_effect(cdf, "balance", "job")
    R["kruskal_age_education"] = kruskal_effect(cdf, "age", "education")
    # multivariate
    lo = logit_odds_ratios(train)
    tables["logit_odds_ratios"] = lo
    R["logit"] = {"pseudo_r2": lo.attrs["pseudo_r2"], "n": lo.attrs["n"]}
    v_before, v_after = vif_table(train, False), vif_table(train, True)
    tables["vif_before"], tables["vif_after"] = v_before.to_frame("VIF"), v_after.to_frame("VIF")
    R["vif"] = {"max
_before": float(v_before.max()), "max_before_name": v_before.idxmax(),
                "n_above10_before": int((v_before > 10).sum()), "max_after": float(v_after.max()),
                "max_after_name": v_after.idxmax(), "n_above10_after": int((v_after > 10).sum())}
    # contact='unknown' is not evenly spread over time: check its overlap with May
    R["contact_unknown_share_in_may"] = float((cdf.loc[cdf["contact"] == "unknown", "month"] == "may").mean())
    R["contact_unknown_share_in_jun"] = float((cdf.loc[cdf["contact"] == "unknown", "month"] == "jun").mean())

    R["tables_index"] = list(tables.keys())
    if save:
        for k, t in tables.items():
            t.to_csv(TAB_DIR / f"{k}.csv")
        (RES_DIR / "eda_results.json").write_text(json.dumps(_clean_json(R), indent=2))
    R["_tables"] = tables
    return R
```

src/visualizations.py

```
"""Stage 5 (Represent) & Stage 6 (Refine): all exploratory figures.

Design rules used everywhere (Stage 6): descriptive titles that state the question, labelled axes
with units, one consistent colour pair for the outcome (blue = no, orange = yes), reference lines
where they aid reading, annotated sample sizes, and no 3-D or dual-axis effects.
"""
import matplotlib
```

```
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats

from .config import FIG_DIR, MONTH_ORDER, SEED
from .eda import CONT_DISTS, age_group, rate_table
from .preprocessing import signed_log1p

C_NO, C_YES, C_GREY, C_ACC = "#4C72B0", "#DD8452", "#8C8C8C", "#2E8B57"
plt.rcParams.update({"figure.dpi": 100, "savefig.dpi": 200, "font.size": 10, "axes.titlesize": 11,
                     "axes.titleweight": "bold", "axes.labelsize": 10, "axes.spines.top": False,
                     "axes.spines.right": False, "axes.grid": True, "grid.alpha": .25,
                     "legend.frameon": False})


def _save(fig, name: str):
    path = FIG_DIR / f"{name}.png"
    fig.savefig(path, bbox_inches="tight", facecolor="white")
    plt.close(fig)
    return path


# ---------------------------------------------------------------------------- Figure 1
def fig01_class_balance(cdf):
    vc = cdf["y"].value_counts().reindex(["no", "yes"])
    fig, ax = plt.subplots(figsize=(5.2, 4))
    bars = ax.bar(["No (declined)", "Yes (subscribed)"], vc.values, color=[C_NO, C_YES], width=.6)
    for b, v in zip(bars, vc.values):
        ax.text(b.get_x() + b.get_width() / 2, v + 400, f"{v:,}\n({100 * v / vc.sum():.1f}%)", ha="center", va="bottom")
    ax.set_ylim(0, vc.max() * 1.18)
    ax.set_ylabel("Number of clients"); ax.set_title("Fig. 1  Outcome of the telemarketing call (term deposit)")
    ax.grid(axis="x", visible=False)
    return _save(fig, "fig01_class_balance")


# ---------------------------------------------------------------------------- Figure 2
def fig02_numeric_histograms(cdf):
    cols = [("age", "Age (years)", None), ("balance", "Average yearly balance (EUR)", .99),
            ("duration", "Last call duration (seconds)", .99), ("campaign", "Contacts in this campaign (count)", None),
            ("previous", "Contacts before this campaign (count)", None), ("day", "Day of month of last contact", None)]
    fig, axes = plt.subplots(2, 3, figsize=(13, 7))
    for ax, (c, lab, clip) in zip(axes.ravel(), cols):
        s = cdf[c]
        note = ""
        if clip:
            hi = s.quantile(clip); note = f"\n(x-axis cut at 99th pct = {hi:,.0f}; {int((s > hi).sum()):,} larger values not
shown)"
            s_plot = s[s <= hi]
        elif c in ("campaign", "previous"):
            hi = 20 if c == "campaign" else 10; s_plot = s[s <= hi]
            note = f"\n(shown up to {hi}; {int((s > hi).sum()):,} larger values not shown)"
        else:
            s_plot = s
        if c in ("campaign", "previous"):
            vc = s_plot.value_counts().sort_index()
            ax.bar(vc.index, vc.values, color=C_NO, width=.8)
        else:
            # integer-valued variables get integer-aligned bins (avoids aliasing gaps)
            bins = np.arange(s_plot.min() - .5, s_plot.max() + 1.5, 1) if c in ("age", "day") else 40
            ax.hist(s_plot, bins=bins, color=C_NO, edgecolor="white", linewidth=.4)
        ax.axvline(s.mean(), color=C_YES, ls="--", lw=1.6, label=f"mean {s.mean():,.1f}")
        ax.axvline(s.median(), color="k", ls=":", lw=1.6, label=f"median {s.median():,.1f}")
        ax.set_xlabel(lab + note, fontsize=8.5); ax.set_ylabel("Number of clients")
        ax.set_title(f"{c}: skew = {stats.skew(s):.2f}"); ax.legend(fontsize=8)
    fig.suptitle("Fig. 2  Univariate distributions of the numeric variables", fontweight="bold", y=1.01)
    fig.tight_layout()
    return _save(fig, "fig02_numeric_histograms")


# ---------------------------------------------------------------------------- Figure 3
def fig03_outlier_boxplots(cdf, outliers):
    specs = [("age", "linear", "Age (years)"), ("balance", "symlog", "Balance (EUR, symlog scale)"),
             ("duration", "log", "Call duration (s, log scale)"), ("campaign", "log", "Contacts in campaign (log scale)"),
             ("previous", "symlog", "Previous contacts (symlog scale)")]
    fig, axes = plt.subplots(1, 5, figsize=(14, 4.6))
    o = outliers.set_index("variable")
    for ax, (c, sc, lab) in zip(axes, specs):
        sub = cdf[cdf[c] > 0] if sc == "log" else cdf
        ax.boxplot(sub[c], whis=1.5, patch_artist=True, boxprops=dict(facecolor="#cfe0f3"),
                   medianprops=dict(color="k"), flierprops=dict(marker=".", markersize=2.5, alpha=.35, markeredgecolor=C_NO))
        if sc == "symlog": ax.set_yscale("symlog", linthresh=(100 if c == "balance" else 1))
        elif sc == "log": ax.set_yscale("log")
        ax.set_xticks([]); ax.set_ylabel(lab)
        ax.set_title(f"{c}\n{o.loc[c, 'pct_outliers']:.1f}% beyond 1.5xIQR\n({int(o.loc[c, 'n_extreme(>3IQR)']):,} beyond
3xIQR)", fontsize=9.5)
    fig.suptitle("Fig. 3  Outlier screening with the IQR rule (whiskers = 1.5 x IQR)", fontweight="bold", y=1.04)
    fig.tight_layout()
    return _save(fig, "fig03_outlier_boxplots")


# ---------------------------------------------------------------------------- Figure 4
def fig04_categorical_counts(cdf):
    cols = ["job", "education", "marital", "contact", "poutcome", "month"]
    fig, axes = plt.subplots(2, 3, figsize=(13, 7.4))
```

```
    for ax, c in zip(axes.ravel(), cols):
        vc = cdf[c].value_counts()
        vc = vc.reindex(MONTH_ORDER) if c == "month" else vc.sort_values()
        colors = [C_GREY if i == "unknown" else C_NO for i in vc.index]
        ax.barh(vc.index.astype(str), vc.values, color=colors)
        for y, v in enumerate(vc.values):
            ax.text(v, y, f" {100 * v / len(cdf):.1f}%", va="center", fontsize=8)
        ax.set_xlim(0, vc.max() * 1.18); ax.set_xlabel("N
umber of clients"); ax.set_title(c)
        ax.grid(axis="y", visible=False)
    fig.suptitle("Fig. 4  Category frequencies (grey bars = 'unknown', a hidden form of missing data)",
                 fontweight="bold", y=1.01)
    fig.tight_layout()
    return _save(fig, "fig04_categorical_counts")


# ---------------------------------------------------------------------------- Figure 5
def fig05_rate_by_category(cdf):
    d = cdf.assign(age_group=age_group(cdf["age"]).astype(str))
    base = d["y_bin"].mean()
    cols = ["job", "poutcome", "contact", "education", "housing", "loan"]
    fig, axes = plt.subplots(2, 3, figsize=(13, 7.4))
    for ax, c in zip(axes.ravel(), cols):
        t = rate_table(d, c).sort_values("rate")
        err = np.vstack([t["rate"] - t["ci_low"], t["ci_high"] - t["rate"]]) * 100
        ax.barh(t.index.astype(str), t["rate"] * 100, xerr=err, color=C_ACC, ecolor="k", capsize=2, alpha=.85)
        for y, (r, n) in enumerate(zip(t["rate"], t["n"])):
            ax.text(t["ci_high"].iloc[y] * 100 + .8, y, f"{100 * r:.1f}%  (n={int(n):,})", va="center", fontsize=7.5)
        ax.axvline(base * 100, color=C_YES, ls="--", lw=1.4)
        ax.set_xlim(0, max(t["ci_high"]) * 100 * 1.55); ax.set_xlabel("Subscription rate (%)")
        ax.set_title(c); ax.grid(axis="y", visible=False)
    axes.ravel()[0].legend([plt.Line2D([], [], color=C_YES, ls="--")], [f"overall {base * 100:.1f}%"], loc="lower right")
    fig.suptitle("Fig. 5  Subscription rate by client / campaign category (bars = rate, whiskers = exact 95% CI)",
                 fontweight="bold", y=1.01)
    fig.tight_layout()
    return _save(fig, "fig05_rate_by_category")


# ---------------------------------------------------------------------------- Figure 6
def fig06_month_volume_vs_rate(cdf):
    t = rate_table(cdf, "month", order=MONTH_ORDER)
    fig, ax = plt.subplots(figsize=(7.5, 5))
    err = np.vstack([t["rate"] - t["ci_low"], t["ci_high"] - t["rate"]]) * 100
    ax.errorbar(t["n"], t["rate"] * 100, yerr=err, fmt="o", color=C_ACC, ecolor=C_GREY, capsize=3, ms=7)
    for m, r in t.iterrows():
        ax.annotate(m.capitalize(), (r["n"], r["rate"] * 100), textcoords="offset points", xytext=(7, 5), fontsize=9)
    ax.axhline(cdf["y_bin"].mean() * 100, color=C_YES, ls="--", lw=1.3, label=f"overall {cdf['y_bin'].mean() * 100:.1f}%")
    ax.set_xscale("log"); ax.set_xlabel("Calls made in the month (log scale)"); ax.set_ylabel("Subscription rate (%)")
    ax.set_title("Fig. 6  Busy months convert worst: call volume vs. subscription rate"); ax.legend()
    return _save(fig, "fig06_month_volume_vs_rate")


# ---------------------------------------------------------------------------- Figure 7
def fig07_count_distribution(cdf, R):
    fits = [("campaign", cdf["campaign"].values - 1, R["campaign_counts"], "Extra contacts in campaign (= campaign - 1)", 15),
            ("previous", cdf["previous"].values, R["previous_counts"], "Contacts before this campaign", 15)]
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
    for ax, (nm, x, f, lab, kmax) in zip(axes, fits):
        k = np.arange(0, kmax + 1)
        obs = np.array([(x == i).mean() for i in k])
        ax.bar(k, obs, color="#cfe0f3", edgecolor=C_NO, label="Observed")
        ax.plot(k, stats.poisson.pmf(k, f["poisson_lambda"]), "o-", color=C_YES, label=f"Poisson
(lambda={f['poisson_lambda']:.2f})")
        ax.plot(k, stats.nbinom.pmf(k, f["nb_r"], f["nb_p"]), "s-", color=C_ACC, label=f"Neg. binomial (r={f['nb_r']:.2f},
p={f['nb_p']:.2f})")
        ax.set_yscale("log"); ax.set_ylim(1e-6, 1); ax.set_xlabel(lab); ax.set_ylabel("Probability (log scale)")
        ax.set_title(f"{nm}: variance/mean = {f['dispersion_index_var_over_mean']:.1f} (Poisson would give 1)")
        ax.legend(fontsize=8.5)
    fig.suptitle("Fig. 7  Discrete distributions: observed count PMF vs. fitted Poisson and Negative Binomial",
                 fontweight="bold", y=1.02)
    fig.tight_layout()
    return _save(fig, "fig07_count_distribution")


# ---------------------------------------------------------------------- Figures 8 & 9
def _fit_overlay(x, tables, names, xmax, xlabel, title, fname, qq_pair, bins=60):
    xs = np.linspace(max(x.min(), 1e-6), xmax, 400)
    fig, axes = plt.subplots(1, 3, figsize=(14.5, 4.6))
    ax = axes[0]
    ax.hist(x[x <= xmax], bins=bins, density=True, color="#cfe0f3", edgecolor="white", label="Observed")
    cols = [C_YES, C_ACC, "#8172B3", "#C44E52"]
    params = {}
    for nm, c in zip(names, cols):
        r = tables[tables.distribution == nm].iloc[0]
        p = r["params"]; params[nm] = p
        ax.plot(xs, CONT_DISTS[nm].pdf(xs, *p), color=c, lw=2, label=f"{nm} (AIC +{r.delta_AIC:,.0f}, KS D={r.KS_D:.3f})")
    ax.set_xlabel(xlabel); ax.set_ylabel("Density"); ax.set_title("Histogram with fitted densities"); ax.legend(fontsize=8)
    for ax, nm in zip(axes[1:], qq_pair):
        q = np.linspace(.005, .995, 200)
        emp = np.quantile(x, q); th = CONT_DISTS[nm].ppf(q, *params[nm])
        ax.scatter(th, emp, s=9, color=C_NO)
        lim = [min(th.min(), emp.min()), max(th.max(), emp.max())]
        ax.plot(lim, lim, "k--", lw=1); ax.set_xlabel(f"Theoretical quantiles ({nm})"); ax.set_ylabel("Empirical quantiles")
        ax.set_title(f"Q-Q plot vs {nm}")
    fig.suptitle(title, fontweight="bold", y=1.03)
    fig.tight_layout()
    return _save(fig, fname)
```

```
def fig08_age_fits(cdf, T):
    return _fit_overlay(cdf["age"].values.astype(float), T["fit_age"], ["normal", "lognormal", "gamma"], 95, "Age (years)",
                        "Fig. 8  Continuous distributions: is age normally distributed?", "fig08_age_fits", ["normal",
"lognormal"], bins=39)


def fig09_duration_fits(cdf, T):
    x = cdf.loc[cdf["duration"] > 0, "duration"].values.astype(float)
    return _fit_overlay(x, T["fit_duration"], ["exponential", "lognormal", "gamma", "weibull"], 1500, "Call duration (seconds,
cut at 1,500)",
                        "Fig. 9  Continuous distri
butions: is call duration a memoryless (exponential) waiting time?",
                        "fig09_duration_fits", ["exponential", "lognormal"])


# ---------------------------------------------------------------------------- Figure 10
def fig10_correlation_heatmaps(T):
    lab = {"y_bin": "subscribed (0/1)", "pdays_clean": "pdays (clean)"}
    fig, axes = plt.subplots(1, 2, figsize=(14, 5.8))
    for ax, k, ttl in zip(axes, ["corr_spearman", "corr_pearson"], ["Spearman rank correlation", "Pearson linear correlation"]):
        m = T[k].rename(index=lab, columns=lab)
        sns.heatmap(m, annot=True, fmt=".2f", cmap="RdBu_r", vmin=-1, vmax=1, center=0, square=True, linewidths=.5,
                    cbar_kws={"shrink": .75, "label": "correlation coefficient"}, ax=ax, annot_kws={"fontsize": 8})
        ax.set_title(ttl)
    fig.suptitle("Fig. 10  Correlation structure of the numeric variables and the outcome", fontweight="bold", y=1.02)
    fig.tight_layout()
    return _save(fig, "fig10_correlation_heatmaps")


# ---------------------------------------------------------------------------- Figure 11
def fig11_cramers_heatmap(T):
    m = T["cramers_matrix"].astype(float)
    mask = np.triu(np.ones_like(m, dtype=bool), k=1)
    fig, ax = plt.subplots(figsize=(8.5, 7))
    sns.heatmap(m, mask=mask, annot=True, fmt=".2f", cmap="viridis", vmin=0, vmax=.5, square=True, linewidths=.5,
                cbar_kws={"shrink": .75, "label": "Cramer's V (bias-corrected)"}, ax=ax, annot_kws={"fontsize": 8})
    ax.set_title("Fig. 11  Association between categorical variables (and the outcome 'y')")
    return _save(fig, "fig11_cramers_heatmap")


# ---------------------------------------------------------------------------- Figure 12
def fig12_pairplot(cdf, n=2000):
    s = cdf.sample(n, random_state=SEED)
    d = pd.DataFrame({"age": s["age"], "balance\n(signed log)": signed_log1p(s["balance"]),
                      "duration\n(log1p s)": np.log1p(s["duration"]), "campaign\n(log1p)": np.log1p(s["campaign"]),
                      "previous\n(log1p)": np.log1p(s["previous"]), "Subscribed": s["y"].map({"no": "No", "yes": "Yes"})})
    g = sns.pairplot(d, hue="Subscribed", palette={"No": C_NO, "Yes": C_YES}, corner=True, diag_kind="kde",
diag_kws={"common_norm": False},
                     plot_kws={"s": 8, "alpha": .45, "edgecolor": "none"}, height=2.1)
    g.figure.suptitle(f"Fig. 12  Pairplot of transformed numeric variables (random sample of {n:,} clients)", fontweight="bold",
y=1.02)
    path = FIG_DIR / "fig12_pairplot.png"
    g.figure.savefig(path, bbox_inches="tight", facecolor="white"); plt.close(g.figure)
    return path


# ---------------------------------------------------------------------------- Figure 13
def fig13_age_and_balance(cdf):
    d = cdf.assign(age_group=age_group(cdf["age"]).astype(str))
    t = rate_table(d, "age_group", order=["<25", "25-34", "35-44", "45-54", "55-64", "65+"])
    fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.8))
    ax = axes[0]
    err = np.vstack([t["rate"] - t["ci_low"], t["ci_high"] - t["rate"]]) * 100
    ax.errorbar(range(len(t)), t["rate"] * 100, yerr=err, fmt="o-", color=C_ACC, capsize=4, lw=2, ms=7)
    for i, (r, n) in enumerate(zip(t["rate"], t["n"])):
        ax.text(i, r * 100 + 3.2, f"n={int(n):,}", ha="center", fontsize=8)
    ax.axhline(cdf["y_bin"].mean() * 100, color=C_YES, ls="--", label="overall rate")
    ax.set_ylim(0, t["ci_high"].max() * 100 + 9)
    ax.set_xticks(range(len(t))); ax.set_xticklabels(t.index); ax.set_xlabel("Age group (years)"); ax.set_ylabel("Subscription
rate (%)")
    ax.set_title("Non-monotonic: the youngest and oldest clients subscribe most"); ax.legend()
    ax = axes[1]
    d2 = pd.DataFrame({"sl": signed_log1p(cdf["balance"]), "Subscribed": cdf["y"].map({"no": "No", "yes": "Yes"})})
    sns.violinplot(data=d2, x="Subscribed", y="sl", hue="Subscribed", palette={"No": C_NO, "Yes": C_YES}, inner="quartile",
                   cut=0, ax=ax, legend=False)
    ticks = np.array([-1000, -100, 0, 100, 1000, 10000, 100000]); ax.set_yticks(signed_log1p(ticks))
    ax.set_yticklabels([f"{t_:,}" for t_ in ticks]); ax.set_ylabel("Balance (EUR, signed-log scale)")
    ax.set_title("Subscribers hold higher balances (but heavily overlap)")
    fig.suptitle("Fig. 13  Age and account balance versus subscription", fontweight="bold", y=1.03)
    fig.tight_layout()
    return _save(fig, "fig13_age_and_balance")


# ---------------------------------------------------------------------------- Figure 14
def fig14_odds_ratio_forest(T, top=16):
    lo = T["logit_odds_ratios"].copy()
    lo["effect"] = np.abs(np.log(lo["odds_ratio"]))
    lo = lo.sort_values("effect", ascending=False).head(top).sort_values("odds_ratio")
    fig, ax = plt.subplots(figsize=(8.5, 6.5))
    ax.errorbar(lo["odds_ratio"], range(len(lo)), xerr=[lo["odds_ratio"] - lo["ci_low"], lo["ci_high"] - lo["odds_ratio"]],
                fmt="o", color=C_ACC, ecolor=C_GREY, capsize=3)
    ax.axvline(1, color="k", ls="--", lw=1); ax.set_xscale("log")
    ax.set_yticks(range(len(lo))); ax.set_yticklabels(lo.index); ax.set_xlabel("Adjusted odds ratio (log scale, 95% CI)")
    ax.set_title("Fig. 14  Multivariable logistic regression: the 16 strongest adjusted associations")
    ax.grid(axis="y", visible=False)
```

```
    return _save(fig, "fig14_odds_ratio_forest")


def make_all_eda_figures(cdf, R):
    T = R["_tables"]
    return {1: fig01_class_balance(cdf), 2: fig02_numeric_histograms(cdf), 3: fig03_outlier_boxplots(cdf, T["outliers_iqr"]),
            4: fig04_categorical_counts(cdf), 5: fig05_rate_by_category(cdf), 6: fig06_month_volume_vs_rate(cdf),
            7: fig07_count_distribution(cdf, R), 8: fig08_age_fits(cdf, T), 9: fig09_duration_fits(cdf, T),
            10: fig10_correlation_heatmaps(T), 11: fig11_cramers_heatmap(T), 12: fig12_pairplot(cdf),
            13: fig13_age_and_balance(cdf), 14: fig14_odds_ratio_forest(T)}
```

src/model.py

```
"""Deep-learning model: a regularised Multi-Layer Perceptron (MLP) for tabular binary classification.

Why an MLP? The data are tabular (numeric + one-hot categorical), so there is no spatial or
sequential structure for a CNN / RNN / Transformer to exploit. An MLP is the natural, defensible
deep-learning choice, and it is compared against classical baselines in `train.py`.
"""
import os

os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "2")
os.environ.setdefault("TF_ENABLE_ONEDNN_OPTS", "0")  # bit-for-bit reproducibility on CPU

import tensorflow as tf  # noqa: E402
from tensorflow import keras  # noqa: E402
from tensorflow.keras import layers, regularizers  # noqa: E402


def set_reproducible(seed: int) -> None:
    """Seed Python, NumPy and TensorFlow and request deterministic kernels."""
    keras.utils.set_random_seed(seed)
    try:
        tf.config.experimental.enable_op_determinism()
    except Exception:  # noqa: BLE001 - older TF builds
        pass


def build_mlp(input_dim: int, units=(64, 32), dropout: float = 0.3, l2: float = 1e-4,
              lr: float = 1e-3) -> keras.Model:
    """Dense(ReLU) -> Dropout blocks followed by a single sigmoid unit (P(subscribe))."""
    inp = keras.Input(shape=(input_dim,), name="features")
    x = inp
    for i, u in enumerate(units):
        x = layers.Dense(u, activation="relu", kernel_regularizer=regularizers.l2(l2),
                         kernel_initializer="he_normal", name=f"dense_{i + 1}")(x)
        x = layers.Dropout(dropout, name=f"dropout_{i + 1}")(x)
    out = layers.Dense(1, activation="sigmoid", name="p_subscribe")(x)
    model = keras.Model(inp, out, name="bank_mlp")
    model.compile(optimizer=keras.optimizers.Adam(learning_rate=lr),
                  loss=keras.losses.BinaryCrossentropy(),
                  metrics=["accuracy", keras.metrics.AUC(name="auc")])
    return model
```

src/train.py

```
"""Step 4: model building - baselines, hyper-parameter search, final multi-seed training,
robustness experiments and permutation importance. Everything is saved to outputs/.

Usage:
    python -m src.train            # full experiment (~15-25 min on a single CPU core)
    python -m src.train --quick    # smoke test only (2 configs, 3 epochs) -> outputs/smoke/
"""
import argparse
import itertools
import json
import platform
import random
import time

import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression

from .config import MODEL_DIR, OUT, RES_DIR, SEED, TAB_DIR
from .data_loader import get_data
from .evaluate import best_f1_threshold, compute_metrics, permutation_importance
from .model import build_mlp, set_reproducible
from .preprocessing import clean, model_columns, prepare_matrices, split_data

import tensorflow as tf  # noqa: E402  (after .model so env flags are set first)
from tensorflow import keras  # noqa: E402

GRID = {"units": [(32,), (64, 32), (128, 64, 32)], "dropout": [0.2, 0.4], "lr": [1e-3, 3e-4],
        "class_weight": [None, "balanced"]}
BATCH, MAX_EPOCHS, PATIENCE = 256, 40, 6


def _cw(y, mode):
    if mode != "balanced":
```

```
        return None
    n, n1 = len(y), int(y.sum())
    return {0: n / (2 * (n - n1)), 1: n / (2 * n1)}


def train_mlp(cfg, Xtr, ytr, Xva, yva, seed, max_epochs=MAX_EPOCHS):
    """Train one MLP with early stopping on validation ROC-AUC; returns (model, history dict)."""
    set_reproducible(seed)
    m = build_mlp(Xtr.shape[1], cfg["units"], cfg["dropout"], 1e-4, cfg["lr"])
    es = keras.callbacks.EarlyStopping(monitor="val_auc", mode="max", patience=PATIENCE, restore_best_weights=True)
    h = m.fit(Xtr, ytr, validation_data=(Xva, yva), epochs=max_epochs, batch_size=BATCH, verbose=0, shuffle=True,
              callbacks=[es], class_weight=_cw(ytr, cfg["class_weight"]))
    return m, {k: [float(v) for v in vs] for k, vs in h.history.items()}


def predict(m, X):
    return m.predict(X, batch_size=2048, verbose=0).ravel()


def _eval_scores(pva, pte, yva, yte):
    thr = best_f1_threshold(yva, pva)
    return {"val": compute_metrics(yva, pva, thr), "test": compute_metrics(yte, pte, thr)}


def run_baselines(Xtr, ytr, Xva, yva, Xte, yte):
    out = {}
    base = np.full(len(yva), ytr.mean()); out["Majority class"] = _eval_scores(base, np.full(len(yte), ytr.mean()), yva, yte)
    lr = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
    out["Logistic regression"] = _eval_scores(lr.predict_proba(Xva)[:, 1], lr.predict_proba(Xte)[:, 1], yva, yte)
    hgb = HistGradientBoostingClassifier(random_state=SEED).fit(Xtr, ytr)
    out["Gradient boosting (reference)"] = _eval_scores(hgb.predict_proba(Xva)[:, 1], hgb.predict_proba(Xte)[:, 1], yva, yte)
    return out


def _cached(path, fn):
    """Resumable runs: reuse a completed step's JSON result if it exists, otherwise compute and save."""
    if path.exists():
        return json.loads(path.read_text())
    val = fn()
    path.write_text(json.dumps(val, default=float))
    return val


def run_experiments(quick: bool = False):
    t0 = time.time()
    out_dir = OUT / "smoke" if quick else OUT
    (out_dir / "models").mkdir(parents=True, exist_ok=True)
    (out_dir / "results").mkdir(parents=True, exist_ok=True)
    mdl_dir, res_dir = out_dir / "models", out_dir / "results"
    cache = res_dir / "_cache"; cache.mkdir(exist_ok=True)

    raw = get_data(); cdf = clean(raw)
    tr, va, te = split_data(cdf, "stratified")
    Xtr, Xva, Xte, ytr, yva, yte, pre, names = prepare_matrices(tr, va, te, include_duration=False)
    assert Xtr.shape[1] == len(names)
    print(f"Matrices: train {Xtr.shape}, val {Xva.shape}, test {Xte.shape}; {len(names)} model inputs", flush=True)

    # ---------------- hyper-parameter search (validation only; test set untouched) ----------------
    combos = [dict(zip(GRID, v)) for v in itertools.product(*GRID.values())]
    n_cfg = 2 if quick else 12
    cfgs = random.Random(SEED).sample(combos, n_cfg)
    max_ep = 3 if quick else MAX_EPOCHS
    rows = []
    for i, cfg in enumerate(cfgs):
        def _one(cfg=cfg, i=i):
            ts = time.time()
            m, h = train_mlp(cfg, Xtr, ytr, Xva, yva, SEED, max_ep)
            mv = compute_metrics(yva, predict(m, Xva), 0.5)
            return {"config_id": i, "units": str(cfg["units"]), "dropout": cfg["dropout"], "lr": cfg["lr"],
                    "class_weight": cfg["class_weight"] or "none", "epochs_run": len(h["loss"]),
                    "best_epoch": int(np.argmax(h["val_auc"])) + 1, "val_roc_auc": mv["roc_auc"],
                    "val_pr_auc": mv["pr_auc"], "seconds": round(time.time() - ts, 1)}
        rows.append(_cached(cache / f"search_{i}.json", _one))
        print(f"[search {i + 1}/{n_cfg}] {cfg} -> val ROC-AUC {rows[-1]['val_roc_auc']:.4f}  PR-AUC {rows[-1]
['val_pr_auc']:.4f}", flush=True)
    search = pd.DataFrame(rows).sort_values("val_pr_auc", ascending=False).reset_index(drop=True)
    best_id = int(search.loc[0, "config_id"]); best_cfg = cfgs[best_id]
    search.to_csv(res_dir / "hparam_search.csv", index=False)
    print("Selected:", best_cfg, flush=True)

    # ---------------- final model: 5 seeds, same config ----------------
    seeds = [SEED] if quick else [42, 43, 44, 45, 46]
    finals = []
    for sd in seeds:
        def _fin(sd=sd):
            m, h = train_mlp(best_cfg, Xtr, ytr, Xva, yva, sd, max_ep)
            pva, pte = predict(m, Xva), predict(m, Xte)
            sc = _eval_scores(pva, pte, yva, yte); sc.update(seed=sd, epochs_run=len(h["loss"]),
best_epoch=int(np.argmax(h["val_auc"])) + 1)
            if sd == SEED:  # keep the primary model's artefacts
                m.save(mdl_dir / "mlp_primary.keras")
                (res_dir / "history_primary.json").write_text(json.dumps(h))
                np.savez(res_dir / "predictions_primary.npz", p_val=pva, p_test=pte, y_val=yva, y_test=
yte)
            return sc
        sc = _cached(cache / f"final_{sd}.json", _fin); finals.append(sc)
        print(f"[final seed {sd}] test ROC-AUC {sc['test']['roc_auc']:.4f} PR-AUC {sc['test']['pr_auc']:.4f} F1 {sc['test']
```

```
['f1']:.4f}", flush=True)
    m = keras.models.load_model(mdl_dir / "mlp_primary.keras"); joblib.dump(pre, mdl_dir / "preprocessor.joblib")
    n_params = int(m.count_params())
    arch = []
    m.summary(print_fn=lambda s: arch.append(s))

    # ---------------- baselines (same pre-processed data) ----------------
    baselines = _cached(cache / "baselines.json", lambda: run_baselines(Xtr, ytr, Xva, yva, Xte, yte))
    np_lr = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
    np.save(res_dir / "p_test_logreg.npy", np_lr.predict_proba(Xte)[:, 1])

    # ---------------- permutation importance (validation set; original input columns) ----------------
    cols = model_columns(False)
    fn = lambda d: m.predict(pre.transform(d[cols]).astype("float32"), batch_size=4096, verbose=0).ravel()  # noqa: E731
    imp, base_auc = permutation_importance(fn, va[cols], yva, cols, n_repeats=2 if quick else 5)
    imp.to_csv(res_dir / "permutation_importance.csv", index=False)

    # ---------------- robustness experiments ----------------
    rob = []
    def add(setting, model, s):
        rob.append({"setting": setting, "model": model, "test_roc_auc": s["test"]["roc_auc"], "test_pr_auc": s["test"]["pr_auc"],
                    "test_f1": s["test"]["f1"], "test_prevalence": s["test"]["prevalence"]})
    arr = lambda key: float(np.mean([f["test"][key] for f in finals]))  # noqa: E731
    rob.append({"setting": "Pre-call features\n(primary, stratified)", "model": "MLP", "test_roc_auc": arr("roc_auc"),
                "test_pr_auc": arr("pr_auc"), "test_f1": arr("f1"), "test_prevalence": float(yte.mean())})
    for k, lab in (("Logistic regression", "Logistic regression"), ("Gradient boosting (reference)", "Gradient boosting")):
        add("Pre-call features\n(primary, stratified)", lab, baselines[k])
    def _dur():
        Xtr2, Xva2, Xte2, ytr2, yva2, yte2, _, _n = prepare_matrices(tr, va, te, include_duration=True)
        m2, _h = train_mlp(best_cfg, Xtr2, ytr2, Xva2, yva2, SEED, max_ep)
        return {"mlp": _eval_scores(predict(m2, Xva2), predict(m2, Xte2), yva2, yte2),
                "base": run_baselines(Xtr2, ytr2, Xva2, yva2, Xte2, yte2)}
    d2 = _cached(cache / "duration.json", _dur); b2 = d2["base"]
    add("+ call duration\n(not known pre-call)", "MLP", d2["mlp"])
    add("+ call duration\n(not known pre-call)", "Logistic regression", b2["Logistic regression"])
    add("+ call duration\n(not known pre-call)", "Gradient boosting", b2["Gradient boosting (reference)"])
    ctr, cva, cte = split_data(cdf, "chronological")
    def _chr():
        Xtr3, Xva3, Xte3, ytr3, yva3, yte3, _, _n = prepare_matrices(ctr, cva, cte, include_duration=False)
        m3, _h = train_mlp(best_cfg, Xtr3, ytr3, Xva3, yva3, SEED, max_ep)
        return {"mlp": _eval_scores(predict(m3, Xva3), predict(m3, Xte3), yva3, yte3),
                "base": run_baselines(Xtr3, ytr3, Xva3, yva3, Xte3, yte3),
                "prev": [float(ytr3.mean()), float(yva3.mean()), float(yte3.mean())]}
    c3 = _cached(cache / "chrono.json", _chr); s3, b3 = c3["mlp"], c3["base"]
    add("Pre-call features,\nchronological split", "MLP", s3)
    add("Pre-call features,\nchronological split", "Logistic regression", b3["Logistic regression"])
    add("Pre-call features,\nchronological split", "Gradient boosting", b3["Gradient boosting (reference)"])
    robustness = pd.DataFrame(rob); robustness.to_csv(res_dir / "robustness.csv", index=False)
    chrono = {"train_prev": c3["prev"][0], "val_prev": c3["prev"][1], "test_prev": c3["prev"][2],
              "mlp": s3, "logreg": b3["Logistic regression"], "hgb": b3["Gradient boosting (reference)"]}

    results = {
        "best_config": {**best_cfg, "units": list(best_cfg["units"])}, "n_params": n_params,
        "n_inputs": int(Xtr.shape[1]), "input_names": names, "split_sizes": [len(tr), len(va), len(te)],
        "split_prevalence": [float(ytr.mean()), float(yva.mean()), float(yte.mean())],
        "batch_size": BATCH, "max_epochs": max_ep, "early_stopping_patience": PATIENCE, "l2": 1e-4,
        "optimizer": "Adam", "loss": "binary cross-entropy", "n_search_configs": n_cfg,
        "primary": {"seed": SEED, **finals[[f["seed"] for f in finals].index(SEED)]},
        "seeds": [f["seed"] for f in finals],
        "test_mean_std": {k: [float(np.mean([f["test"][k] for f in finals])), float(np.std([f["test"][k] for f in finals]))]
                          for k in ("roc_auc", "pr_auc", "f1", "precision", "recall", "accuracy", "balanced_accuracy", "mcc",
"brier")},
        "all_seed_runs": finals, "baselines": baselines, "baselines_with_duration": b2,
        "duration_ablation_mlp": rob[3], "chronological": chrono, "permutation_base_auc": base_auc,
        "architecture_summary": arch, "quick_smoke_test": quick,
        "environment": {"python": platform.python_version(), "tensorflow": tf.__version__, "platform": platform.platform(),
                        "cpu_count": __import__("os").cpu_count(), "gpu": [d.name for d in
tf.config.list_physical_devices("GPU")]},
        "runtime_seconds": round(time.time() - t0, 1),
    }
    (res_dir / "model_results.json").write_text(json.dumps(results, indent=2, default=float))
    print(f"Done in {results['runtime_seconds']} s. Saved to {res_dir}", flush=True)
    return results


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true", help="smoke test only (NOT a full training run)")
    run_experiments(ap.parse_args().quick)
```

src/evaluate.py

```
"""Model evaluation: metrics, threshold selection, permutation importance and result figures."""
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.metrics import (accuracy_score, average_precision_score, balanced_accuracy_score,
                             brier_score_loss, confusion_matrix, f1_score, matthews_corrcoef,
                             precision_recall_curve, precision_score, recall_score, roc_auc_score,
                             roc_curve)
```

```
from .config import FIG_DIR
from .visualizations import C_ACC, C_GREY, C_NO, C_YES, _save


# ---------------------------------------------------------------------------- metrics
def best_f1_threshold(y_true, p) -> float:
    """Threshold maximising F1 on the VALIDATION set (never tuned on the test set)."""
    prec, rec, thr = precision_recall_curve(y_true, p)
    f1 = 2 * prec[:-1] * rec[:-1] / np.clip(prec[:-1] + rec[:-1], 1e-12, None)
    return float(thr[int(np.argmax(f1))])


def compute_metrics(y_true, p, thr: float) -> dict:
    yhat = (np.asarray(p) >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, yhat).ravel()
    return {"threshold": thr, "accuracy": accuracy_score(y_true, yhat),
            "balanced_accuracy": balanced_accuracy_score(y_true, yhat),
            "precision": precision_score(y_true, yhat, zero_division=0),
            "recall": recall_score(y_true, yhat), "f1": f1_score(y_true, yhat),
            "mcc": matthews_corrcoef(y_true, yhat), "roc_auc": roc_auc_score(y_true, p),
            "pr_auc": average_precision_score(y_true, p), "brier": brier_score_loss(y_true, p),
            "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
            "prevalence": float(np.mean(y_true))}


def permutation_importance(predict_fn, X_df: pd.DataFrame, y, columns, n_repeats=5, seed=42):
    """Drop in ROC-AUC when one ORIGINAL input column is shuffled (one-hot groups move together)."""
    rng = np.random.default_rng(seed)
    base = roc_auc_score(y, predict_fn(X_df))
    rows = []
    for c in columns:
        drops = []
        for _ in range(n_repeats):
            Xp = X_df.copy()
            Xp[c] = rng.permutation(Xp[c].values)
            drops.append(base - roc_auc_score(y, predict_fn(Xp)))
        rows.append({"feature": c, "auc_drop_mean": float(np.mean(drops)), "auc_drop_std": float(np.std(drops))})
    return pd.DataFrame(rows).sort_values("auc_drop_mean", ascending=False).reset_index(drop=True), base


# ---------------------------------------------------------------------------- figures
def fig15_hparam_search(search: pd.DataFrame, best_idx: int):
    s = search.sort_values("val_pr_auc").reset_index(drop=True)
    labels = [f"{r.units} | do={r.dropout} | lr={r.lr} | cw={r.class_weight}" for r in s.itertuples()]
    fig, ax = plt.subplots(figsize=(10, 0.42 * len(s) + 1.6))
    colors = [C_ACC if r.config_id == best_idx else C_NO for r in s.itertuples()]
    ax.barh(range(len(s)), s["val_pr_auc"], color=colors)
    for i, (a, b) in enumerate(zip(s["val_pr_auc"], s["val_roc_auc"])):
        ax.text(a + .002, i, f"PR-AUC {a:.3f} | ROC-AUC {b:.3f}", va="center", fontsize=8)
    ax.set_yticks(range(len(s))); ax.set_yticklabels(labels, fontsize=8)
    ax.set_xlim(s["val_pr_auc"].min() * .9, s["val_pr_auc"].max() * 1.12)
    ax.set_xlabel("Validation PR-AUC (selection criterion)")
    ax.set_title("Fig. 15  Hyper-parameter search (validation set only; green = selected)")
    ax.grid(axis="y", visible=False)
    return _save(fig, "fig15_hparam_search")


def fig16_loss_curves(hist):
    ep = np.arange(1, len(hist["loss"]) + 1)
    fig, ax = plt.subplots(figsize=(7.2, 4.6))
    ax.plot(ep, hist["loss"], color=C_NO, lw=2, label="Training loss")
    ax.plot(ep, hist["val_loss"], color=C_YES, lw=2, label="Validation loss")
    b = int(np.argmin(hist["val_loss"])) + 1
    ax.axvline(b, color=C_GREY, ls=":", label=f"lowest val. loss (epoch {b})")
    ax.set_xlabel("Epoch"); ax.set_ylabel("Binary cross-entropy (+ L2 penalty)")
    ax.set_title("Fig. 16  Training vs. validation loss"); ax.legend()
    return _save(fig, "fig16_loss_curves")


def fig17_accuracy_curves(hist, majority_acc):
    ep = np.arange(1, len(hist["accuracy"]) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
    axes[0].plot(ep, np.array(hist["accuracy"]) * 100, color=C_NO, lw=2, label="Training")
    axes[0].plot(ep, np.array(hist["val_accuracy"]) * 100, color=C_YES, lw=2, label="Validation")
    axes[0].axhline(majority_acc * 100, color="k", ls="--", lw=1.2, label=f"always-'no' baseline {majority_acc * 100:.1f}%")
    axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Accuracy at 0.5 threshold (%)"); axes[0].set_title("Accuracy");
axes[0].legend()
    axes[1].plot(ep, hist["auc"], color=C_NO, lw=2, label="Training")
    axes[1].plot(ep, hist["val_auc"], color=C_YES, lw=2, label="Validation")
    axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("ROC-AUC"); axes[1].set_title("ROC-AUC (threshold-free)"); axes[1].legend()
    fig.suptitle("Fig. 17  Accuracy and AUC versus epoch (accuracy alone is misleading at 11.7% prevalence)", fontweight="bold",
y=1.02)
    fig.tight_layout()
    return _save(fig, "fig17_accuracy_auc_curves")


def fig18_confusion(m):
    cm = np.array([[m["tn"], m["fp"]], [m["fn"], m["tp"]]])
    pct = cm / cm.sum(axis=1, keepdims=True) * 100
    fig, ax = plt.subplots(figsize=(5.8, 5))
    ax.imshow(pct, cmap="Blues", vmin=0, vmax=100)
    for i in range(2):
        for j in range(2):
            ax.text(j, i, f"{cm[i, j]:,}\n({pct[i, j]:.1f}% of row)", ha="center", va="center",
                    color="white" if pct[i, j] > 55 else "black", fontsize=11)
    ax.set_xticks([0, 1]); ax.set_xticklabels(["Predicted: no", "Predicted: yes"])
```

```
    ax.set_yticks([0, 1]); ax.set_yticklabels(["Actual: no", "Actual: yes"]); ax.grid(False)
    ax.set_title(f"Fig. 18  Confusion matrix on the TEST set\n(threshold {m['threshold']:.3f} chosen o
n validation)")
    return _save(fig, "fig18_confusion_matrix")


def fig19_roc_pr(y, curves: dict):
    """curves: name -> (probabilities, colour)."""
    fig, axes = plt.subplots(1, 2, figsize=(12.5, 5))
    for name, (p, col) in curves.items():
        fpr, tpr, _ = roc_curve(y, p)
        axes[0].plot(fpr, tpr, color=col, lw=2, label=f"{name} (AUC = {roc_auc_score(y, p):.3f})")
        pr, rc, _ = precision_recall_curve(y, p)
        axes[1].plot(rc, pr, color=col, lw=2, label=f"{name} (AP = {average_precision_score(y, p):.3f})")
    axes[0].plot([0, 1], [0, 1], "k--", lw=1, label="chance (AUC = 0.5)")
    axes[0].set_xlabel("False-positive rate"); axes[0].set_ylabel("True-positive rate (recall)"); axes[0].set_title("ROC curve");
axes[0].legend(loc="lower right")
    axes[1].axhline(np.mean(y), color="k", ls="--", lw=1, label=f"prevalence = {np.mean(y):.3f}")
    axes[1].set_xlabel("Recall"); axes[1].set_ylabel("Precision"); axes[1].set_title("Precision-recall curve");
axes[1].legend(loc="upper right")
    axes[1].set_ylim(0, 1.02)
    fig.suptitle("Fig. 19  ROC and precision-recall curves on the TEST set", fontweight="bold", y=1.02)
    fig.tight_layout()
    return _save(fig, "fig19_roc_pr_curves")


def fig20_permutation_importance(imp: pd.DataFrame, base_auc: float):
    d = imp.sort_values("auc_drop_mean")
    fig, ax = plt.subplots(figsize=(8, 5.6))
    ax.barh(d["feature"], d["auc_drop_mean"], xerr=d["auc_drop_std"], color=C_ACC, ecolor="k", capsize=2)
    ax.set_xlabel(f"Drop in validation ROC-AUC when the feature is shuffled (baseline AUC = {base_auc:.3f})")
    ax.set_title("Fig. 20  Permutation feature importance of the MLP (5 repeats)")
    ax.grid(axis="y", visible=False)
    return _save(fig, "fig20_permutation_importance")


def fig21_threshold_tradeoff(y_val, p_val, thr):
    prec, rec, th = precision_recall_curve(y_val, p_val)
    f1 = 2 * prec[:-1] * rec[:-1] / np.clip(prec[:-1] + rec[:-1], 1e-12, None)
    fig, ax = plt.subplots(figsize=(7.2, 4.6))
    ax.plot(th, prec[:-1], color=C_NO, lw=2, label="Precision"); ax.plot(th, rec[:-1], color=C_YES, lw=2, label="Recall")
    ax.plot(th, f1, color=C_ACC, lw=2, label="F1")
    ax.axvline(thr, color="k", ls=":", label=f"chosen threshold {thr:.3f}")
    ax.set_xlabel("Decision threshold on P(subscribe)"); ax.set_ylabel("Score (validation set)")
    ax.set_title("Fig. 21  Precision / recall / F1 versus decision threshold"); ax.legend()
    return _save(fig, "fig21_threshold_tradeoff")


def fig22_robustness(rows: pd.DataFrame):
    """rows: columns setting, model, test_roc_auc, test_pr_auc."""
    settings = list(dict.fromkeys(rows["setting"]))
    models = list(dict.fromkeys(rows["model"]))
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
    w = .8 / len(models)
    cols = [C_YES, C_NO, C_ACC, "#8172B3"]  # same colours as Fig. 19 (MLP orange, logistic blue)
    for ax, metric, lab in zip(axes, ["test_roc_auc", "test_pr_auc"], ["Test ROC-AUC", "Test PR-AUC"]):
        for i, m in enumerate(models):
            vals = [rows[(rows.setting == s) & (rows.model == m)][metric].mean() for s in settings]
            bars = ax.bar(np.arange(len(settings)) + i * w, vals, w, label=m, color=cols[i % 4])
            for b, v in zip(bars, vals):
                ax.text(b.get_x() + b.get_width() / 2, v + .005, f"{v:.3f}", ha="center", fontsize=8)
        ax.set_xticks(np.arange(len(settings)) + w * (len(models) - 1) / 2); ax.set_xticklabels(settings, fontsize=9)
        ax.set_ylabel(lab); ax.legend(fontsize=8.5, loc="upper left")
        ax.set_ylim(0, 1.08 if "roc" in metric else max(rows[metric].max() * 1.2, .5)); ax.grid(axis="x", visible=False)
    fig.suptitle("Fig. 22  Robustness: the MLP and baselines under different feature sets and split protocols",
                 fontweight="bold", y=1.02)
    fig.tight_layout()
    return _save(fig, "fig22_robustness")


# ---------------------------------------------------------------------------- driver
def make_model_figures(res_dir=None):
    """Regenerate every model-evaluation figure from the artifacts saved by `src.train`."""
    import json

    from .config import RES_DIR
    res_dir = res_dir or RES_DIR
    R = json.loads((res_dir / "model_results.json").read_text())
    search = pd.read_csv(res_dir / "hparam_search.csv")
    hist = json.loads((res_dir / "history_primary.json").read_text())
    z = np.load(res_dir / "predictions_primary.npz")
    imp = pd.read_csv(res_dir / "permutation_importance.csv")
    rob = pd.read_csv(res_dir / "robustness.csv")
    p_lr = np.load(res_dir / "p_test_logreg.npy")
    thr = R["primary"]["val"]["threshold"]
    m_test = compute_metrics(z["y_test"], z["p_test"], thr)
    figs = {15: fig15_hparam_search(search, int(search.loc[0, "config_id"])),
            16: fig16_loss_curves(hist), 17: fig17_accuracy_curves(hist, 1 - R["split_prevalence"][2]),
            18: fig18_confusion(m_test),
            19: fig19_roc_pr(z["y_test"], {"MLP (primary, seed 42)": (z["p_test"], C_YES), "Logistic regression": (p_lr, C_NO)}),
            20: fig20_permutation_importance(imp, R["permutation_base_auc"]),
            21: fig21_threshold_tradeoff(z["y_val"], z["p_val"], thr), 22: fig22_robustness(rob)}
    return figs, m_test
```

src/interactive.py

```
"""Stage 7 (Interact): a self-contained interactive HTML explorer built with Plotly.

Open outputs/interactive/explorer.html in any browser (needs internet only to load plotly.js from a CDN).
"""
import numpy as np
import pandas as pd
import plotly.graph_objects as go

from .config import CATEGORICAL, MONTH_ORDER, OUT, SEED
from .eda import rate_table

C_NO, C_YES, C_ACC = "#4C72B0", "#DD8452", "#2E8B57"


def _rate_fig(cdf):
    cdf = cdf.copy()
    fig = go.Figure()
    for i, c in enumerate(CATEGORICAL):
        t = rate_table(cdf, c, order=MONTH_ORDER if c == "month" else None)
        if c != "month":
            t = t.sort_values("rate", ascending=False)
        fig.add_bar(x=t.index.astype(str), y=t["rate"] * 100, visible=(i == 0), marker_color=C_ACC, name=c,
                    error_y=dict(type="data", symmetric=False, array=(t["ci_high"] - t["rate"]) * 100,
                                 arrayminus=(t["rate"] - t["ci_low"]) * 100),
                    customdata=np.c_[t["n"], t["subscribed"]],
                    hovertemplate="<b>%{x}</b><br>rate %{y:.1f}%<br>n = %{customdata[0]:,}<br>subscribed = %
{customdata[1]:,}<extra></extra>")
    buttons = [dict(label=c, method="update", args=[{"visible": [j == i for j in range(len(CATEGORICAL))]},
                                                    {"title": f"Subscription rate by {c} (95% exact CI)"}])
               for i, c in enumerate(CATEGORICAL)]
    fig.add_hline(y=cdf["y_bin"].mean() * 100, line_dash="dash", line_color=C_YES,
                  annotation_text=f"overall {cdf['y_bin'].mean() * 100:.1f}%")
    fig.update_layout(title=f"Subscription rate by {CATEGORICAL[0]} (95% exact CI)", yaxis_title="Subscription rate (%)",
                      updatemenus=[dict(buttons=buttons, x=0.0, y=1.18, xanchor="left")], template="plotly_white", height=470)
    return fig


def _hist_fig(cdf):
    specs = [("age", "age (years)"), ("balance", "balance (EUR, clipped at 1st-99th pct)"),
             ("duration", "duration (s, clipped at 99th pct)"), ("campaign", "campaign contacts (clipped at 99th pct)"),
             ("previous", "previous contacts (clipped at 99th pct)"), ("day", "day of month")]
    fig = go.Figure()
    for i, (c, lab) in enumerate(specs):
        s = cdf[c]
        lo, hi = (s.quantile(.01), s.quantile(.99)) if c in ("balance",) else (s.min(), s.quantile(.99) if c in ("duration",
"campaign", "previous") else s.max())
        for yv, col, nm in (("no", C_NO, "No"), ("yes", C_YES, "Yes")):
            v = cdf.loc[(cdf["y"] == yv) & (s >= lo) & (s <= hi), c]
            fig.add_histogram(x=v, histnorm="probability density", opacity=.6, marker_color=col, name=nm,
                              visible=(i == 0), legendgroup=nm, showlegend=(i == 0), nbinsx=50)
    buttons = [dict(label=c, method="update", args=[{"visible": [(j // 2) == i for j in range(2 * len(specs))],
                                                     "showlegend": [(j // 2) == i for j in range(2 * len(specs))]},
                                                    {"title": f"Distribution of {lab} by outcome", "xaxis": {"title": lab}}])
               for i, (c, lab) in enumerate(specs)]
    fig.update_layout(barmode="overlay", title=f"Distribution of {specs[0][1]} by outcome", xaxis_title=specs[0][1],
                      yaxis_title="Density", updatemenus=[dict(buttons=buttons, x=0.0, y=1.18, xanchor="left")],
                      template="plotly_white", height=470)
    return fig


def _scatter_fig(cdf, n=3000):
    s = cdf.sample(n, random_state=SEED)
    fig = go.Figure()
    for yv, col, nm in (("no", C_NO, "No"), ("yes", C_YES, "Yes")):
        d = s[s["y"] == yv]
        fig.add_scattergl(x=d["age"], y=d["duration"], mode="markers", marker=dict(color=col, size=6, opacity=.55), name=nm,
                          customdata=np.c_[d["job"], d["month"], d["balance"], d["campaign"], d["poutcome"]],
                          hovertemplate=("age %{x}, call %{y} s<br>job: %{customdata[0]}<br>month: %{customdata[1]}"
                                         "<br>balance: EUR %{customdata[2]:,}<br>contacts: %{customdata[3]}<br>previous outcome:
%{customdata[4]}<extra>" + nm + "</extra>"))
    fig.update_layout(title=f"Age vs. call duration (random sample of {n:,} clients; hover for details, drag to zoom)",
                      xaxis_title="Age (years)", yaxis_title="Call duration (s, log scale)", yaxis_type="log",
template="plotly_white", height=470)
    fig.update_yaxes(rangemode="tozero") if False else None
    return fig


def build_explorer(cdf: pd.DataFrame, path=None):
    path = path or OUT / "interactive" / "explorer.html"
    parts = [_rate_fig(cdf), _hist_fig(cdf), _scatter_fig(cdf[cdf["duration"] > 0])]
    html = ["<html><head><meta charset='utf-8'><title>Bank Marketing - interactive explorer</title>"
            "<style>body{font-family:Arial,sans-serif;max-width:1000px;margin:24px auto;padding:0 12px}</style></head><body>",
            "<h2>Bank Marketing (UCI #222) - interactive explorer</h2>",
            "<p>Stage 7 of the seven-stage visualization framework (Interact). Use the dropdowns, hover for exact values, "
            "drag to zoom and double-click to reset. Source: Moro, Cortez &amp;amp; Rita (2014), UCI Machine Learning
Repository.</p>"]
    for i, f in enumerate(parts):
        html.append(f.to_html(full_html=False, include_plotlyjs="cdn" if i == 0 else False))
    html.append("</body></html>")
    path.write_text("\n".join(html), encoding="utf-8")
    return path
```

scripts/run_all.py

```
"""Run the whole pipeline end to end:  python scripts/run_all.py [--skip-train]

1 Acquire -> 2 EDA/statistics -> 3 EDA figures -> 4 DL experiments (resumable) -> 5 model figures
-> 6 interactive explorer -> 7 notebook + report.
"""
import argparse
import sys
import warnings
from pathlib import Path

warnings.filterwarnings("ignore")
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))

from src.data_loader import get_data  # noqa: E402
from src.eda import run_eda  # noqa: E402
from src.interactive import build_explorer  # noqa: E402
from src.preprocessing import clean, split_data  # noqa: E402
from src.visualizations import make_all_eda_figures  # noqa: E402

ap = argparse.ArgumentParser()
ap.add_argument("--skip-train", action="store_true", help="reuse saved model artefacts in outputs/results")
ap.add_argument("--quick", action="store_true", help="smoke-test the training code only (not a full run)")
a = ap.parse_args()

raw = get_data(); cdf = clean(raw); train, val, test = split_data(cdf)
R = run_eda(raw, cdf, train)
print("EDA figures:", len(make_all_eda_figures(cdf, R)))
if not a.skip_train:
    from src.train import run_experiments
    run_experiments(quick=a.quick)
if not a.quick:
    from src.evaluate import make_model_figures
    figs, _ = make_model_figures(); print("Model figures:", len(figs))
print("Explorer:", build_explorer(cdf))
```